



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΙΑΣ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Μελέτη Απόδοσης και Κλιμάκωσης Συστήματος RAG με Μοντέλα Γλώσσας σε Υποδομή Kubernetes

Φοιτητής

Θεοφιλογιαννάκος Βασίλειος

ΥΠΕΥΘΥΝΟΣ

Κωνσταντίνου Ιωάννης

Λαμία, Ιούνιος 2026



UNIVERSITY OF
THESSALY

SCHOOL OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE & TELECOMMUNICATIONS

Performance and Scaling Study of a RAG System
with
Language Models on Kubernetes Infrastructure

STUDENT

Theofilogiannakos Vasileios

FINAL THESIS

ADVISOR

Konstantinou Ioannis

Lamia, June 2026

«Με ατομική μου ευθύνη και γνωρίζοντας τις κυρώσεις ⁽¹⁾, που προβλέπονται από της διατάξεις της παρ. 6 του άρθρου 22 του Ν. 1599/1986, δηλώνω ότι:

1. Δεν παραθέτω κομμάτια βιβλίων ή άρθρων ή εργασιών άλλων αυτολεξεί **χωρίς να τα περικλείω σε εισαγωγικά** και χωρίς να αναφέρω το συγγραφέα, τη χρονολογία, τη σελίδα. Η αυτολεξεί παράθεση χωρίς εισαγωγικά χωρίς αναφορά στην πηγή, είναι λογοκλοπή. Πέραν της αυτολεξεί παράθεσης, λογοκλοπή θεωρείται και η παράφραση εδαφίων από έργα άλλων, συμπεριλαμβανομένων και έργων συμφοιτητών μου, καθώς και η παράθεση στοιχείων που άλλοι συνέλεξαν ή επεξεργάστηκαν, χωρίς αναφορά στην πηγή. Αναφέρω πάντοτε με πληρότητα την πηγή κάτω από τον πίνακα ή σχέδιο, όπως στα παραθέματα.

2. Δέχομαι ότι η αυτολεξεί **παράθεση χωρίς εισαγωγικά**, ακόμα κι αν συνοδεύεται από αναφορά στην πηγή σε κάποιο άλλο σημείο του κειμένου ή στο τέλος του, είναι αντιγραφή. Η αναφορά στην πηγή στο τέλος π.χ. μιας παραγράφου ή μιας σελίδας, δεν δικαιολογεί συρραφή εδαφίων έργου άλλου συγγραφέα, έστω και παραφρασμένων, και παρουσίασή τους ως δική μου εργασία.

3. Δέχομαι ότι υπάρχει επίσης περιορισμός στο μέγεθος και στη συχνότητα των παραθεμάτων που μπορώ να εντάξω στην εργασία μου εντός εισαγωγικών. Κάθε μεγάλο παράθεμα (π.χ. σε πίνακα ή πλαίσιο, κλπ), προϋποθέτει ειδικές ρυθμίσεις, και όταν δημοσιεύεται προϋποθέτει την άδεια του συγγραφέα ή του εκδότη. Το ίδιο και οι πίνακες και τα σχέδια

4. Δέχομαι όλες τις συνέπειες σε περίπτωση λογοκλοπής ή αντιγραφής.

Ημερομηνία: 29/6/2026

Ο –

Η Δηλ.

Θεοφιλογιαννάκος Βασίλειος

(1) «Όποιος εν γνώσει του δηλώνει ψευδή γεγονότα ή αρνείται ή αποκρύπτει τα αληθινά με έγγραφη υπεύθυνη δήλωση του άρθρου 8 παρ. 4 Ν. 1599/1986 τιμωρείται με φυλάκιση τουλάχιστον τριών μηνών. Εάν ο υπαίτιος αυτών των πράξεων σκόπευε να προσπορίσει στον εαυτόν του ή σε άλλον περιουσιακό όφελος βλάπτοντας τρίτον ή σκόπευε να βλάψει άλλον, τιμωρείται με κάθειρξη μέχρι 10 ετών.»

Περίληψη

Η αυξανόμενη αξιοποίηση των Μεγάλων Γλωσσικών Μοντέλων (Large Language Models, LLMs) σε εφαρμογές παραγωγικής λειτουργίας δημιουργεί σημαντικές απαιτήσεις ως προς την εξυπηρέτηση μοντέλων, τη διαχείριση υπολογιστικών πόρων και την αυτόματη κλιμάκωση. Η παρούσα πτυχιακή εργασία μελετά τη σχεδίαση, την υλοποίηση και την πειραματική αξιολόγηση ενός συστήματος Παραγωγής Επαυξημένης με Ανάκτηση (Retrieval-Augmented Generation, RAG) σε υποδομή Kubernetes, με έμφαση στην αυτόματη κλιμάκωση, την ψυχρή εκκίνηση και την απόδοση σε υποδομή αποκλειστικής χρήσης κεντρικών μονάδων επεξεργασίας (CPU). Αναπτύχθηκε ο γνωσιακός βοηθός Atlas Systems, βασισμένος σε συνθετική εταιρική γνωσιακή βάση. Το σύστημα αποτελείται από υπηρεσία FastAPI, διανυσματική βάση δεδομένων ChromaDB, Ollama για την εξυπηρέτηση των μοντέλων και διεπαφή χρήστη Gradio. Η γνωσιακή βάση περιλαμβάνει δεδομένα για υπαλλήλους, έργα, τμήματα, πολιτικές, τεχνική υποστήριξη, συναντήσεις, μηνιαία πλάνα και αγγελίες θέσεων εργασίας. Αξιολογήθηκαν τα μοντέλα Phi-3 Mini και Mistral 7B μέσω δοκιμών φόρτου με το Locust.

Η πειραματική αξιολόγηση έδειξε ότι ο μηχανισμός Οριζόντιας Αυτόματης Κλιμάκωσης Pods (Horizontal Pod Autoscaler, HPA) μπορεί να λειτουργεί σωστά ως προς την αύξηση του αριθμού των αντιγράφων, χωρίς όμως αυτό να συνεπάγεται άμεσα διαθέσιμη νέα χωρητικότητα εξυπηρέτησης. Η ψυχρή εκκίνηση ενός νέου αντιγράφου περιλαμβάνει πολλαπλά στάδια, όπως τη διαθεσιμότητα της εικόνας περιέκτη, τη διαθεσιμότητα του μοντέλου στον δίσκο, τη φόρτωσή του στην κύρια μνήμη και την αρχική προθέρμανση. Για τον μετριασμό αυτών των καθυστερήσεων υλοποιήθηκε συνδυασμός προδημιουργημένων εικόνων περιεκτών, προθέρμανσης κόμβων μέσω DaemonSet, μηχανισμού postStart, ανιχνευτή ετοιμότητας και ρύθμισης OLLAMA_KEEP_ALIVE. Επιπλέον, η αφαίρεση της

αναγκαστικής επανεισαγωγής δεδομένων από τον περιέκτη αρχικοποίησης μείωσε τον χρόνο ψυχρής εκκίνησης του Mistral, σε κόμβο με ήδη διαθέσιμη εικόνα του Backend, από περίπου 6,5 λεπτά σε περίπου 2,8 λεπτά.

Τα αποτελέσματα δείχνουν ότι το Phi-3 Mini επιτυγχάνει σημαντικά χαμηλότερη καθυστέρηση απόκρισης, υψηλότερο ρυθμό εξυπηρέτησης και μικρότερη κατανάλωση μνήμης σε σχέση με το Mistral 7B στην εξεταζόμενη υποδομή CPU. Και τα δύο μοντέλα παρείχαν ορθές απαντήσεις σε ερωτήματα απλής ανάκτησης πραγματολογικών πληροφοριών. Ωστόσο, το Mistral 7B παρουσίασε πληρέστερη συμπεριφορά σε περιπτώσεις εξαγωγής λιστών, ελλιπούς πληροφορίας και επακόλουθων ερωτημάτων. Συνολικά, για τον συγκεκριμένο φόρτο εργασίας RAG, το Phi-3 Mini αποτελεί καταλληλότερη επιλογή, καθώς συνδυάζει επαρκή ποιότητα απαντήσεων με σαφώς αποδοτικότερη λειτουργία σε περιβάλλον αποκλειστικής χρήσης CPU.

Λέξεις-κλειδιά: Kubernetes, Μεγάλα Γλωσσικά Μοντέλα, Παραγωγή Επαυξημένη με Ανάκτηση, αυτόματη κλιμάκωση, ψυχρή εκκίνηση, υποδομή CPU, ChromaDB, Ollama, Phi-3 Mini, Mistral 7B.

Abstract

The increasing adoption of Large Language Models (LLMs) in production applications creates significant demands on serving infrastructure — particularly in CPU-only environments where high inference latency and slow replica startup impose serious limitations. This thesis studies the design, performance and scalability of a Retrieval-Augmented Generation (RAG) system deployed on Kubernetes, focusing on the challenges of elastic autoscaling and cold-start mitigation for LLM workloads.

An enterprise knowledge assistant, Atlas Systems, was developed based on a synthetic corporate knowledge base of 447 documents. The system consists of a FastAPI service, a ChromaDB vector database, and a co-located Ollama container serving quantised open-source models (Phi-3 Mini and Mistral 7B). A five-layer cold-start mitigation strategy was designed and implemented, combining pre-baked container images, DaemonSet-based image pre-warming, conditional data ingestion, postStart lifecycle hooks and custom readiness probes.

Experimental evaluation demonstrated that the Horizontal Pod Autoscaler (HPA) functions correctly as a replica-creation mechanism but is not sufficient on its own for LLM workloads: newly-created replicas require significant time before they can serve real traffic. The proposed mitigation strategy substantially reduces pod ready-time from several minutes to under 90 seconds. A comparative evaluation between Phi-3 Mini and Mistral 7B under Locust load testing showed that the smaller model achieves 4.7× higher throughput and roughly 3× lower p95 latency on CPU-only infrastructure, while providing adequate quality for factual retrieval — making it the more suitable choice for the specific workload.

Keywords: Kubernetes, Large Language Models, Retrieval-Augmented Generation, horizontal autoscaling, cold start, CPU-only infrastructure, ChromaDB, Ollama, Phi-3 Mini, Mistral 7B.

Ευχαριστίες

Θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή μου, κύριο Κωνσταντίνου Ιωάννη, για τη συνεχή καθοδήγηση, τις πολύτιμες συμβουλές του καθ' όλη τη διάρκεια εκπόνησης αυτής της πτυχιακής εργασίας, καθώς και για την παροχή πρόσβασης στο υπολογιστικό σύμπλεγμα `vdcloud(cluster)` που αποτέλεσε την πειραματική πλατφόρμα για την αξιολόγηση του συστήματος.

Τέλος, ευχαριστώ την οικογένειά μου για την υποστήριξη και την υπομονή τους κατά τη διάρκεια των σπουδών μου.

Περιεχόμενα

Περίληψη	4
Abstract	6
Ευχαριστίες.....	7
Περιεχόμενα.....	8
Πίνακας Εικόνων	11
Πίνακας Πινάκων.....	13
Γλωσσάρι Τεχνικών Όρων	14
Βασικές έννοιες μοντέλων και RAG	14
Περιέκτες, Kubernetes και διαχείριση πόρων.....	15
Ψυχρή εκκίνηση, λειτουργία και αξιολόγηση.....	15
ΚΕΦΑΛΑΙΟ 1 Εισαγωγή.....	17
1.1 Κίνητρο και Πλαίσιο της Εργασίας.....	17
1.2 Επιλογή Υποδομής Αποκλειστικής Χρήσης Κεντρικών Μονάδων Επεξεργασίας (CPU-only)	17
1.3 Στόχοι της Εργασίας.....	18
1.4 Συνεισφορά.....	19
1.5 Δομή της Εργασίας.....	20
ΚΕΦΑΛΑΙΟ 2 Θεωρητικό Υπόβαθρο	21
2.1 Μεγάλα Γλωσσικά Μοντέλα (LLMs)	21
2.1.1 Η αρχιτεκτονική Transformer	21
2.1.2 Κβάντιση: Συμπίεση για εξαγωγή συμπερασμάτων σε CPU.....	22
2.1.3 Τα δύο μοντέλα της σύγκρισης	22
2.2 Παραγωγή Επαυξημένη με Ανάκτηση (Retrieval-Augmented Generation, RAG)	23
2.2.1 Διανυσματικές Αναπαραστάσεις (embeddings) και Διανυσματικές Βάσεις Δεδομένων.....	24
2.3 Περιέκτες και Kubernetes	25

2.3.1 Βασικά Αντικείμενα Kubernetes	26
2.3.2 Μηχανισμός Οριζόντιας Αυτόματης Κλιμάκωσης (HPA)	26
2.3.3 Αυτήματα και Όρια Πόρων	27
2.4 Το Πρόβλημα της Ψυχρής Εκκίνησης στην Εξαγωγή Συμπερασμάτων από LLM.....	27
ΚΕΦΑΛΑΙΟ 3 Σχετική Έρευνα και Εργαλεία	29
3.1 Πλαίσια Εξυπηρέτησης Μεγάλων Γλωσσικών Μοντέλων	29
3.2 Πλαίσια Λογισμικού και Βάσεις Δεδομένων για RAG	30
3.3 Αυτόματη Κλιμάκωση σε Kubernetes.....	30
ΚΕΦΑΛΑΙΟ 4 Σχεδίαση του Συστήματος.....	32
4.1 Επισκόπηση Αρχιτεκτονικής.....	32
4.2 Επιλογή Γνωσιακής Βάσης — Atlas Systems	33
4.3 Επιλογή Συνόλου Τεχνολογιών	34
4.4 Διάταξη Παράπλευρου Περιέκτη (FastAPI + Ollama)	34
4.5 Περιέκτης Αρχικοποίησης (Init Container) για Εισαγωγή Δεδομένων (ingestion).....	35
4.6 Μοτίβο Προδημιουργημένης Εικόνας (Pre-Baked Image)	35
4.7 Μοτίβο Προθέρμανσης Κόμβων μέσω DaemonSet (DaemonSet Pre-Warm).....	36
4.8 Μηχανισμός postStart και Προσαρμοσμένος Ανιχνευτής Ετοιμότητας (Custom Readiness Probe)	38
4.9 Ρύθμιση OLLAMA_KEEP_ALIVE — Αντιστάθμιση μεταξύ Προθερμασμένων Αντιγράφων (Warm Pool) και Πίεσης Μνήμης....	39
4.10 Διαμόρφωση HPA.....	41
4.11 Πολυμοντελική Αρχιτεκτονική για Συγκριτική Μελέτη.....	41
ΚΕΦΑΛΑΙΟ 5 Υλοποίηση.....	43
5.1 Οργάνωση του Project	43
5.2 Παραγωγή του Συνθετικού Συνόλου Δεδομένων (Synthetic Dataset) Atlas Systems	44
5.3 FastAPI Backend (app/main.py)	45

5.4 Διαδικασία RAG (app/rag.py)	45
5.5 Διεπαφή Χρήστη Gradio (ui/gradio_app.py).....	46
5.6 Δημιουργία Εικόνων Περιεκτών (Containerization)	48
5.7 Αρχεία Δηλωτικής Διαμόρφωσης Kubernetes (Manifests)	49
5.8 Makefile	49
5.9 Ρύθμιση Locust	50
ΚΕΦΑΛΑΙΟ 6 Πειραματική Αξιολόγηση	52
6.1 Πειραματική Διάταξη	52
6.2 Λειτουργική Επιβεβαίωση του RAG	53
6.3 Δοκιμές Φόρτου: Phi-3 Mini vs Mistral 7B	54
6.3.1 Μεθοδολογία Καθαρού Σημείου Εκκίνησης (Baseline)	55
6.3.2 Αποτελέσματα: 3 ταυτόχρονοι χρήστες.....	55
6.3.3 Αποτελέσματα: 5 ταυτόχρονοι χρήστες.....	57
6.3.4 Αποτελέσματα: 10 ταυτόχρονοι χρήστες	61
6.3.5 Παρατηρήσεις για τον Κορεσμό του Ρυθμού Εξυπηρέτησης και την Άνιση Κατανομή Φορτίου	64
6.4 Συμπεριφορά του HPA.....	64
6.4.1 Παρατηρήσεις σχετικά με την κατανομή φορτίου	65
6.4.2 Ανατομία Ψυχρής Εκκίνησης (Cold-Start Anatomy)	66
6.4.3 Βελτιστοποίηση Ψυχρής Εκκίνησης: Διόρθωση Επανεισαγωγής Δεδομένων από τον Περιέκτη Αρχικοποίησης.....	67
6.5 Συμπεριφορά του OLLAMA_KEEP_ALIVE υπό Φόρτο	69
6.6 Σύγκριση Ποιότητας Απαντήσεων Phi-3 Mini vs Mistral 7B	70
6.6.1 Εμπειρική Σύγκριση	70
6.6.2 Δημοσιευμένες Δοκιμασίες Αξιολόγησης.....	71
6.6.3 Σύνθεση και Σύσταση	72
6.7 Πρακτικά Σημεία Προσοχής σε Παραγωγική Λειτουργία (Production Gotchas)	73
6.8 Σύνοψη Παρατηρήσεων και Περιορισμοί.....	74

ΚΕΦΑΛΑΙΟ 7 Συμπεράσματα και Μελλοντική Εργασία.....	75
7.1 Σύνοψη Βασικών Ευρημάτων	75
7.2 Διδάγματα Παραγωγής	76
7.3 Μελλοντικές Κατευθύνσεις.....	77
Βιβλιογραφία	78

Πίνακας Εικόνων

Εικόνα 1: Διάγραμμα ροής του συστήματος Παραγωγής Επαυξημένης με Ανάκτηση (RAG).	25
Εικόνα 2: Αρχιτεκτονική του συστήματος.	33
Εικόνα 3: Διάταξη παράπλευρου περιέκτη FastAPI και Ollama στο pod του Backend.	35
Εικόνα 4: Μηχανισμός προθέρμανσης κόμβων μέσω DaemonSet.	37
Εικόνα 5: Ανάπτυξη του DaemonSet προθέρμανσης κόμβων.	38
Εικόνα 6: Ανιχνευτής ετοιμότητας (readinessProbe) του περιέκτη Ollama.	39
Εικόνα 7: Μηχανισμός postStart για την προθέρμανση του μοντέλου.	39
Εικόνα 8: Διεπαφή χρήστη Gradio.	47
Εικόνα 9: Ανακτημένα τμήματα κειμένου από τη διανυσματική βάση δεδομένων.....	47
Εικόνα 10: Στατιστικά δοκιμών φόρτου στο Locust.....	51
Εικόνα 11: Έξοδος της εντολής kubectl get pods -n vtheofil-priv πριν από τη δοκιμή φόρτου.	52
Εικόνα 12: Λειτουργική επιβεβαίωση του συστήματος RAG.	54
Εικόνα 13: Παράδειγμα ανακτημένου τμήματος κειμένου.....	54
Εικόνα 14: Στατιστικά Locust για Phi-3 Mini, δοκιμή P1 με 3 ταυτόχρονους χρήστες και διάρκεια 10 λεπτών.	56
Εικόνα 15: Στατιστικά Locust για Mistral 7B, δοκιμή M1 με 3 ταυτόχρονους χρήστες και διάρκεια 10 λεπτών.	57
Εικόνα 16: Στατιστικά και διαγράμματα Locust για Phi-3 Mini, δοκιμή P2 με 5 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.	58
Εικόνα 17: Χρήση πόρων των pods κατά τη δοκιμή Phi-3 P2 μέσω της εντολής kubectl top pods.	59

Εικόνα 18: Στατιστικά Locust για Mistral 7B με 5 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.....	60
Εικόνα 19: Χρήση πόρων των pods κατά τη δοκιμή Mistral 7B με 5 ταυτόχρονους χρήστες μέσω της εντολής <code>kubectl top pods</code>	60
Εικόνα 20: Στατιστικά και διαγράμματα Locust για Mistral 7B, δοκιμή M3 με 10 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.....	62
Εικόνα 21: Στατιστικά και διαγράμματα Locust για Phi-3 Mini, δοκιμή P3 με 10 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.....	63
Εικόνα 22: Κατανομή αιτημάτων συνομιλίας μεταξύ πέντε ενεργών pods.....	66
Εικόνα 23: Χρονική ακολουθία ψυχρής εκκίνησης νέου pod.	68

Πίνακας Πινάκων

Πίνακας 1 Σύγκριση Phi-3 Mini και Mistral 7B στα 3 ταυτόχρονους χρήστες (CPU-only).	56
Πίνακας 2 Σύγκριση Phi-3 Mini και Mistral 7B στα 5 ταυτόχρονους χρήστες, διάρκεια 15 λεπτών.	57
Πίνακας 3 Σύγκριση Phi-3 Mini και Mistral 7B στα 10 ταυτόχρονους χρήστες — το breaking point του Mistral.	61
Πίνακας 4 Ποιοτική Σύγκριση Phi-3 Mini και Mistral 7B.....	71
Πίνακας 5 Δημοσιευμένα benchmark scores Phi-3 Mini vs Mistral 7B (Phi-3 Technical Report, Microsoft 2024 [3]).	72

Γλωσσάρι Τεχνικών Όρων

Στο παρόν γλωσσάρι καταγράφονται οι βασικοί τεχνικοί όροι που χρησιμοποιούνται στην εργασία και η ελληνική απόδοση που υιοθετήθηκε. Κατά την πρώτη εμφάνιση ενός όρου στο κείμενο, η ελληνική απόδοση ακολουθείται από τον αγγλικό όρο σε παρένθεση. Στις επόμενες εμφανίσεις χρησιμοποιείται η ελληνική απόδοση ή η καθιερωμένη συντομογραφία.

Οι επίσημες ονομασίες λογισμικού, μοντέλων, αντικειμένων Kubernetes, μετρικών, παραμέτρων ρυθμίσεων, εντολών και αναγνωριστικών κώδικα διατηρούνται στα αγγλικά, όπως Kubernetes, Docker, FastAPI, Ollama, ChromaDB, Gradio, Locust, Phi-3 Mini, Mistral 7B, pod, Deployment, DaemonSet, ConfigMap, Secret, Service, PVC, postStart, OLLAMA_KEEP_ALIVE και kubectl.

Βασικές έννοιες μοντέλων και RAG

Αγγλικός όρος / συντομογραφία	Ελληνική απόδοση
Large Language Model (LLM)	Μεγάλο Γλωσσικό Μοντέλο
Large Language Models (LLMs)	Μεγάλα Γλωσσικά Μοντέλα
Retrieval-Augmented Generation (RAG)	Παραγωγή Επαυξημένη με Ανάκτηση
Inference	Εξαγωγή συμπερασμάτων
Model serving	Εξυπηρέτηση μοντέλου
Knowledge base	Γνωσιακή βάση
Synthetic dataset	Συνθετικό σύνολο δεδομένων
Metadata	Μεταδεδομένα
Embeddings	Διανυσματικές αναπαραστάσεις
Vector database	Διανυσματική βάση δεδομένων
Retrieval	Ανάκτηση
Factual retrieval	Ανάκτηση πραγματολογικών πληροφοριών
Chunk / chunks	Τμήμα / τμήματα κειμένου
Chunking	Κατάτμηση κειμένου
Top-k retrieval	Ανάκτηση των k συναφέστερων τμημάτων
Prompt	Προτροπή προς το μοντέλο
Context	Πλαίσιο
Hallucination	Παραίσθηση
Knowledge cutoff	Όριο γνώσης
Fine-tuning	Προσαρμογή μοντέλου
Quantization	Κβάντιση
Transformer architecture	Αρχιτεκτονική Transformer
Self-attention	Μηχανισμός αυτοπροσοχής
Autoregressive generation	Αυτοπαλινδρομική παραγωγή
Benchmark	Δοκιμασία αξιολόγησης
Chain of Thought (CoT)	Αλυσίδα σκέψης

Open-weight model	Μοντέλο ανοικτών βαρών
Framework	Πλαίσιο ανάπτυξης
Cloud-native	Νεφοεγγενής
Edge computing	Υπολογιστική στην παρυφή του δικτύου

Περιέκτες, Kubernetes και διαχείριση πόρων

Αγγλικός όρος / συντομογραφία	Ελληνική απόδοση
Central Processing Unit (CPU)	Κεντρική Μονάδα Επεξεργασίας
Graphics Processing Unit (GPU)	Μονάδα Γραφικής Επεξεργασίας
CPU-only infrastructure	Υποδομή αποκλειστικής χρήσης CPU
Container	Περιέκτης
Containerization	Περιεκτοποίηση
Container image	Εικόνα περιέκτη
Pre-baked image	Προδημιουργημένη εικόνα περιέκτη
Cluster	Υπολογιστικό σύμπλεγμα
Node	Κόμβος
Namespace	Χώρος ονομάτων
Pod	Μονάδα εκτέλεσης Kubernetes
Deployment	Αντικείμενο διαχείρισης αντιγράφων εφαρμογής
Replica	Αντίγραφο
Service	Υπηρεσία δικτύου Kubernetes
ConfigMap	Αντικείμενο ρυθμίσεων Kubernetes
Secret	Αντικείμενο αποθήκευσης ευαίσθητων δεδομένων
PersistentVolumeClaim (PVC)	Αίτημα μόνιμου αποθηκευτικού χώρου
Init Container	Περιέκτης αρχικοποίησης
Sidecar	Παράπλευρος περιέκτης
DaemonSet	Αντικείμενο Kubernetes με ένα pod ανά κόμβο
postStart hook	Ενέργεια που εκτελείται μετά την εκκίνηση περιέκτη
Readiness probe	Ανιχνευτής ετοιμότητας
Liveness probe	Ανιχνευτής λειτουργικότητας
Resource requests	Αιτήματα πόρων
Resource limits	Όρια πόρων
Workload	Φόρτος εργασίας
Horizontal Pod Autoscaler (HPA)	Μηχανισμός οριζόντιας αυτόματης κλιμάκωσης pods
Autoscaling	Αυτόματη κλιμάκωση
Scale-up	Κλιμάκωση προς τα πάνω

Ψυχρή εκκίνηση, λειτουργία και αξιολόγηση

Αγγλικός όρος / συντομογραφία	Ελληνική απόδοση
Cold start	Ψυχρή εκκίνηση
Cold-start mitigation	Μετριασμός ψυχρής εκκίνησης
Image pull	Λήψη εικόνας περιέκτη
Model pull	Λήψη μοντέλου
Model load	Φόρτωση μοντέλου στην κύρια μνήμη
Image cache	Τοπική προσωρινή μνήμη εικόνων
Warm-up	Προθέρμανση
DaemonSet pre-warm	Προθέρμανση κόμβων μέσω DaemonSet
Warm pool	Σύνολο προθερμασμένων αντιγράφων
OLLAMA_KEEP_ALIVE	Παράμετρος διατήρησης του μοντέλου στη μνήμη
KV cache	Κρυφή μνήμη κλειδών-τιμών
OOMKill	Τερματισμός περιέκτη λόγω υπέρβασης ορίου μνήμης
Ingestion	Εισαγωγή δεδομένων
Re-ingestion	Επανεισαγωγή δεδομένων
Race condition	Συνθήκη ανταγωνισμού
Load testing	Δοκιμές φόρτου
Baseline	Βασική γραμμή αναφοράς
Latency	Καθυστερήση απόκρισης
Median latency	Διάμεση καθυστέρηση
p95 / p99 latency	95ο / 99ο εκατοστημόριο καθυστέρησης
Throughput	Ρυθμός εξυπηρέτησης
Requests per second (RPS)	Αιτήματα ανά δευτερόλεπτο
Failure rate	Ποσοστό αποτυχιών
Burst load	Αιχμιαίος φόρτος
Sustained load	Συνεχής φόρτος
Follow-up questions	Επακόλουθα ερωτήματα
List extraction	Εξαγωγή στοιχείων λίστας
Missing-information handling	Χειρισμός ελλιπούς πληροφορίας
Structured retrieval	Δομημένη ανάκτηση
Post-processing	Μετεπεξεργασία
L7 request routing	Δρομολόγηση αιτημάτων επιπέδου εφαρμογής
Event-driven scaling	Κλιμάκωση βάσει γεγονότων
Continuous batching	Συνεχής ομαδοποίηση αιτημάτων

ΚΕΦΑΛΑΙΟ 1 Εισαγωγή

1.1 Κίνητρο και Πλαίσιο της Εργασίας

Τα Μεγάλα Γλωσσικά Μοντέλα (Large Language Models, LLMs) έχουν αλλάξει το τοπίο της εφαρμοσμένης τεχνητής νοημοσύνης την τελευταία τριετία. Μοντέλα ανοιχτού κώδικα όπως τα Llama, Mistral και Phi-3 επιτρέπουν σε ομάδες εκτός των μεγάλων τεχνολογικών εταιρειών να αναπτύξουν λειτουργικά συστήματα παραγωγής βασισμένα σε γλωσσικά μοντέλα, χωρίς εξάρτηση από ιδιόκτητες διεπαφές προγραμματισμού εφαρμογών (Application Programming Interfaces, APIs).

Η μετάβαση όμως από ένα πρωτότυπο σύστημα LLM σε ένα σύστημα παραγωγής εισάγει μια κατηγορία προκλήσεων που η ακαδημαϊκή βιβλιογραφία αργεί να καλύψει συστηματικά: πώς εξυπηρετείται ένα γλωσσικό μοντέλο σε διαμοιραζόμενη υπολογιστική υποδομή, πώς κλιμακώνεται όταν αυξάνεται το φορτίο, και πώς συμπεριφέρεται όταν το διαθέσιμο υλικό υπολογιστή (hardware) δεν είναι GPU αλλά απλοί CPU κόμβοι υπολογιστικού συμπλέγματος (cluster).

1.2 Επιλογή Υποδομής Αποκλειστικής Χρήσης Κεντρικών Μονάδων Επεξεργασίας (CPU-only)

Όλα τα πειράματα της παρούσας εργασίας πραγματοποιήθηκαν σε υποδομή αποκλειστικά CPU. Συγκεκριμένα, το υπολογιστικό σύμπλεγμα vndcloud (cluster) του Τμήματος Πληροφορικής και Τηλεπικοινωνιών δεν διέθετε εγγυημένη GPU διαθεσιμότητα κατά τη διάρκεια εκπόνησης. Παρά την αρχική σκέψη να αντιμετωπιστεί αυτή η συνθήκη ως περιορισμός, αξιοποιήθηκε στην πορεία ως πραγματικό αναλυτικό πλεονέκτημα:

- Σε υποδομή αποκλειστικής χρήσης CPU, τα προβλήματα ελαστικότητας γίνονται εντονότερα. Ένα μοντέλο 7 δισεκατομμυρίων παραμέτρων χρειάζεται δευτερόλεπτα έως λεπτά για να παράγει μία απάντηση, αντί για χιλιοστά του

δευτερολέπτου. Συνεπώς, ο σωστός χειρισμός του φόρτου είναι κρίσιμος, όχι προαιρετικός.

- Πολλά πραγματικά περιβάλλοντα παραγωγής, όπως υποδομές υπολογιστικής στην παρυφή του δικτύου (edge computing), τοπικά υπολογιστικά συμπλέγματα μικρών επιχειρήσεων και εκπαιδευτικά ιδρύματα, στερούνται πόρων μονάδων γραφικής επεξεργασίας (GPU). Τα ευρήματα της εργασίας έχουν άμεση εφαρμογή σε αυτά τα σενάρια.
- Τα μοτίβα μετριασμού της ψυχρής εκκίνησης, αυτόματης κλιμάκωσης και διαχείρισης πόρων που αναπτύσσονται εδώ παραμένουν χρήσιμα και σε περιβάλλοντα GPU, με τις αναγκαίες προσαρμογές.

Όλες οι μετρήσεις στα κεφάλαια που ακολουθούν αφορούν τη διάταξη αποκλειστικής χρήσης CPU.

1.3 Στόχοι της Εργασίας

Οι στόχοι της εργασίας ορίζονται ως εξής:

- Σχεδίαση και υλοποίηση ενός ολοκληρωμένου συστήματος Παραγωγής Επαυξημένης με Ανάκτηση (Retrieval-Augmented Generation, RAG), το οποίο εκτελείται σε υπολογιστικό σύμπλεγμα Kubernetes, χρησιμοποιώντας ως ρεαλιστικό φορτίο εργασίας έναν συνθετικό εταιρικό γνωσιακό βοηθό ("Atlas Systems").
- Συστηματική μελέτη της συμπεριφοράς του μηχανισμού οριζόντιας αυτόματης κλιμάκωσης μονάδων εκτέλεσης (Horizontal Pod Autoscaler, HPA) σε φορτία γλωσσικών μοντέλων, με σαφή διαχωρισμό ανάμεσα στη μηχανική σωστή λειτουργία του βρόχου ελέγχου (control loop) και τη συστημική απόδοση του scale-up.
- Σχεδίαση και αξιολόγηση μοτίβων αντιμετώπισης του προβλήματος ψυχρής εκκίνησης (cold start), όπως

προδημιουργημένες εικόνες περιεκτών (pre-baked images), προθέρμανση των κόμβων μέσω DaemonSet, μηχανισμοί postStart για την προθέρμανση του μοντέλου και προσαρμοσμένοι ανιχνευτές ετοιμότητας (readiness probes) που εξαρτώνται από τη διαθεσιμότητα του μοντέλου.

- Συγκριτική μελέτη δύο διαφορετικών μεγάλων γλωσσικών μοντέλων, των Phi-3 Mini και Mistral 7B, στην ίδια υποδομή, με συστηματικό έλεγχο συγχυτικών παραγόντων (confounding factors) που θα μπορούσαν να επηρεάσουν τη σύγκριση, όπως η διαθεσιμότητα της εικόνας περιέκτη στην τοπική προσωρινή μνήμη και η φόρτωση του μοντέλου στην κύρια μνήμη (image cache, model load).

1.4 Συνεισφορά

Η παρούσα εργασία συνεισφέρει με τους ακόλουθους τρόπους:

- Πρακτική απόδειξη ότι η βασική διαμόρφωση του μηχανισμού HPA με κατώφλι βασισμένο στη χρήση της CPU είναι μηχανικά λειτουργική, αλλά συστημικά ανεπαρκής για την εξυπηρέτηση αιτημάτων μεγάλων γλωσσικών μοντέλων (LLM serving) σε υποδομή αποκλειστικής χρήσης CPU, με ποσοτικά στοιχεία από πραγματικά πειράματα.
- Ανάλυση της ψυχρής εκκίνησης (cold start) ως πολυσταδιακού φαινομένου: λήψη της εικόνας περιέκτη (image pull), διαθεσιμότητα του μοντέλου στον δίσκο, φόρτωση του μοντέλου στην κύρια μνήμη (model load) και αρχική προθέρμανση μέσω της πρώτης εξαγωγής συμπερασμάτων (first-inference warmup). Κάθε στάδιο μετράται ξεχωριστά.
- Συνδυαστικό μοτίβο μετριασμού του προβλήματος της ψυχρής εκκίνησης, το οποίο περιλαμβάνει προδημιουργημένη εικόνα περιέκτη (pre-baked image), προθέρμανση κόμβων μέσω DaemonSet (DaemonSet pre-warm), μηχανισμό postStart για την προθέρμανση του μοντέλου, προσαρμοσμένο ανιχνευτή

ετοιμότητας και σύνολο προθερμασμένων αντιγράφων (warm pool) μέσω της ρύθμισης OLLAMA_KEEP_ALIVE.

- Άμεση συγκριτική μελέτη των Phi-3 Mini και Mistral 7B σε υποδομή αποκλειστικής χρήσης CPU, με προφορτωμένες εικόνες περιεκτών σε όλους τους κόμβους, ώστε η σύγκριση να μην επηρεάζεται από την καθυστέρηση λήψης της εικόνας (image pull latency).
- Επιβεβαίωση ότι η ποιότητα των απαντήσεων ενός συστήματος Παραγωγής Επαυξημένης με Ανάκτηση (Retrieval-Augmented Generation, RAG) εξαρτάται ουσιαστικά από τη δομή των δεδομένων και την ποιότητα των μεταδεδομένων, όχι μόνο από την επιλογή του μεγάλου γλωσσικού μοντέλου.

1.5 Δομή της Εργασίας

Η εργασία οργανώνεται ως εξής: Στο Κεφάλαιο 2 παρουσιάζεται το θεωρητικό υπόβαθρο. Στο Κεφάλαιο 3 ανασκοπείται η σχετική έρευνα και τα διαθέσιμα πλαίσια ανάπτυξης. Στο Κεφάλαιο 4 περιγράφεται η σχεδίαση του συστήματος και οι αρχιτεκτονικές αποφάσεις. Στο Κεφάλαιο 5 παρουσιάζεται η τεχνική υλοποίηση. Στο Κεφάλαιο 6 περιγράφονται τα πειράματα και τα αποτελέσματα. Στο Κεφάλαιο 7 συνοψίζονται τα συμπεράσματα και προτείνονται μελλοντικές κατευθύνσεις.

ΚΕΦΑΛΑΙΟ 2 Θεωρητικό Υπόβαθρο

Στο κεφάλαιο αυτό παρουσιάζονται οι έννοιες που είναι απαραίτητες για την κατανόηση της εργασίας. Δεδομένου ότι το κύριο αντικείμενο είναι η υποδομή και όχι η εσωτερική λειτουργία των γλωσσικών μοντέλων, η παρουσίαση των LLMs περιορίζεται στα όσα χρειάζονται για να δικαιολογηθούν οι επιλογές υλοποίησης. Αντίστοιχα, η Kubernetes ενότητα είναι εκτεταμένη και επικεντρώνεται στους μηχανισμούς που χρησιμοποιούνται στη συνέχεια.

2.1 Μεγάλα Γλωσσικά Μοντέλα (LLMs)

Τα LLMs είναι νευρωνικά δίκτυα που εκπαιδεύονται σε μεγάλους όγκους κειμένου ώστε να μάθουν τη στατιστική δομή της γλώσσας και να παράγουν συνεκτικό κείμενο. Σχεδόν όλα τα σύγχρονα LLMs βασίζονται στην αρχιτεκτονική Transformer, η οποία παρουσιάστηκε από τους Vaswani et al. το 2017 [1].

2.1.1 Η αρχιτεκτονική Transformer

Ο Transformer βασίζεται στον μηχανισμό αυτοπροσοχής (self-attention), μέσω του οποίου κάθε λέξη εισόδου μπορεί να αντλεί πληροφορία ταυτόχρονα από όλες τις άλλες λέξεις της ίδιας ακολουθίας. Σε αντίθεση με τη σειριακή επεξεργασία των αναδρομικών νευρωνικών δικτύων (Recurrent Neural Networks, RNN) και των δικτύων μακράς βραχυπρόθεσμης μνήμης (Long Short-Term Memory, LSTM), η αρχιτεκτονική αυτή επιτρέπει την αποτελεσματικότερη αποτύπωση μακρινών εξαρτήσεων στο κείμενο [1]. Τα σύγχρονα LLMs χρησιμοποιούν συνήθως μόνο το αποκωδικοποιητικό τμήμα (decoder) του Transformer, σε αυτοπαλινδρομική διάταξη (autoregressive): παράγουν το επόμενο διακριτό τμήμα κειμένου (token) με βάση όλο το προηγούμενο κείμενο.

2.1.2 Κβάντιση: Συμπίεση για εξαγωγή συμπερασμάτων σε CPU

Ένα μοντέλο 7 δισεκατομμυρίων παραμέτρων αποθηκευμένο σε 16-bit floating-point κατέχει περίπου 14 GB. Σε υποδομές αποκλειστικής χρήσης CPU όπου η RAM είναι ο κύριος περιορισμένος πόρος, αυτή η απαίτηση γίνεται απαγορευτική.

Η κβάντιση (quantization) είναι μια οικογένεια τεχνικών που μειώνουν το μέγεθος ενός μοντέλου αναπαριστώντας τα βάρη του σε χαμηλότερη ακρίβεια. Στην παρούσα εργασία χρησιμοποιείται η μορφή κβάντισης Q4_K_M μέσω της μηχανής llama.cpp [8] (την οποία τρέχει εσωτερικά το Ollama [9]). Με μορφή Q4_K_M, τα βάρη αναπαριστώνται κατά μέσο όρο με περίπου 4,5 bit ανά παράμετρο, μειώνοντας το μέγεθος του μοντέλου περίπου κατά τέσσερις φορές, με μικρή απώλεια ποιότητας. Έτσι, το Mistral 7B καταλαμβάνει στον δίσκο περίπου 4.4 GB αντί για ~14 GB, και το Phi-3 Mini περίπου 2.3 GB αντί για ~7.6 GB.

2.1.3 Τα δύο μοντέλα της σύγκρισης

Η εργασία συγκρίνει δύο μοντέλα ανοικτών βαρών, τα οποία εκτελούνται αμφότερα σε υποδομή αποκλειστικής χρήσης CPU:

- Phi-3 Mini (3,8 δισεκατομμύρια παράμετροι) [3]. Δημοσιεύθηκε από τη Microsoft Research το 2024. Το μέγεθός του στον δίσκο, σε μορφή κβάντισης Q4_K_M, είναι περίπου 2,3 GB.
- Mistral 7B (περίπου 7,3 δισεκατομμύρια παράμετροι) [4]. Αποτελεί το πρώτο μοντέλο γενικής χρήσης της Mistral AI, το οποίο κυκλοφόρησε το 2023 με διαθέσιμα βάρη και άδεια Apache 2.0. Το μέγεθός του σε μορφή Q4_K_M είναι περίπου 4,4 GB.

Η διαφορά κλίμακας μεταξύ των δύο μοντέλων είναι σημαντική για το αντικείμενο της εργασίας: το Phi-3 Mini έχει σχεδόν τις μισές παραμέτρους σε σχέση με το Mistral 7B (3,8 δισεκατομμύρια έναντι περίπου 7,3 δισεκατομμυρίων), γεγονός που μειώνει σημαντικά τις απαιτήσεις σε κύρια μνήμη και αποθηκευτικό χώρο (περίπου 2,3 GB έναντι 4,4 GB στην κβαντισμένη μορφή) και επιτρέπει αποτελεσματικότερη εκτέλεση σε νεφοϋπολογιστικά περιβάλλοντα αποκλειστικής χρήσης CPU.

.

2.2 Παραγωγή Επαυξημένη με Ανάκτηση (Retrieval-Augmented Generation, RAG)

Το RAG, που προτάθηκε από τους Lewis et al. το 2020 [2], συνδυάζει τη γνώση που είναι ενσωματωμένη στις παραμέτρους ενός γλωσσικού μοντέλου με τη δυναμική ανάκτηση πληροφοριών από εξωτερικές πηγές. Σε ένα τυπικό σύστημα RAG, τα έγγραφα της βάσης γνώσης μετατρέπονται σε διανυσματικές αναπαραστάσεις (embeddings) και αποθηκεύονται σε μια διανυσματική βάση δεδομένων. Κατά την υποβολή ενός ερωτήματος, το ερώτημα του χρήστη μετατρέπεται επίσης σε διάνυσμα, ανακτώνται τα k συναφέστερα τμήματα κειμένου (top- k chunks) και προστίθενται στο κείμενο εισόδου προς το μοντέλο (prompt) ως πρόσθετο πλαίσιο (context) πριν από την αποστολή του στο LLM.

Το RAG μπορεί να μετριάσει τρία κύρια προβλήματα των συστημάτων που βασίζονται αποκλειστικά σε LLM: (α) το χρονικό όριο γνώσης (knowledge cutoff), δηλαδή την αδυναμία αξιοποίησης πληροφοριών που προέκυψαν μετά την ολοκλήρωση της εκπαίδευσης του μοντέλου, (β) τις παραισθήσεις (hallucinations), όταν το μοντέλο καλείται να απαντήσει σε ερωτήματα για τα οποία δεν διαθέτει επαρκείς

πληροφορίες στα δεδομένα εκπαίδευσής του, και (γ) τη δυσκολία αξιοποίησης ιδιωτικών ή νεότερων δεδομένων χωρίς προσαρμογή του μοντέλου (fine-tuning).

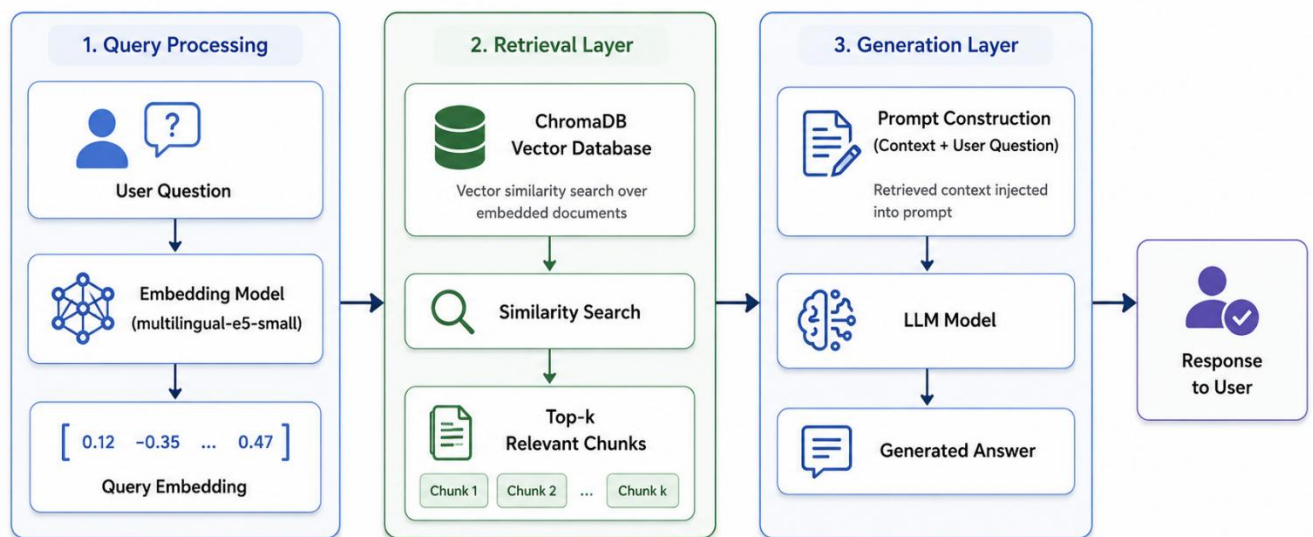
2.2.1 Διανυσματικές Αναπαραστάσεις (embeddings) και Διανυσματικές Βάσεις Δεδομένων

Οι διανυσματικές αναπαραστάσεις μετατρέπουν προτάσεις ή έγγραφα σε διανύσματα σταθερής διάστασης, έτσι ώστε σημασιολογικά παρόμοιο περιεχόμενο να αντιστοιχεί σε κοντινά διανύσματα στον διανυσματικό χώρο. Η εργασία «Sentence-BERT» των Reimers και Gurevych [12] καθιέρωσε τη χρήση μοντέλων Transformer ως κωδικοποιητών προτάσεων. Στην παρούσα εργασία χρησιμοποιείται το μοντέλο `intfloat/multilingual-e5-small` [13], το οποίο είναι πολυγλωσσικό, υποστηρίζει αγγλικά και ελληνικά, έχει μέγεθος περίπου 470 MB, κατάλληλο για εξαγωγή συμπερασμάτων σε CPU, και ενσωματώνεται στην εικόνα περιέκτη Docker της υπηρεσίας υποστήριξης (backend) κατά τη δημιουργία της εικόνας, αποφεύγοντας τη λήψη εξαρτήσεων κατά τον χρόνο εκτέλεσης.

Τα παραγόμενα διανύσματα αποθηκεύονται σε μια διανυσματική βάση δεδομένων που υποστηρίζει κατά προσέγγιση αναζήτηση πλησιέστερων γειτόνων (Approximate Nearest Neighbor search, ANN). Στην παρούσα εργασία χρησιμοποιείται το ChromaDB [10], ένα σύστημα ανοικτού κώδικα που εκτελείται ως αυτόνομη υπηρεσία σε ξεχωριστό Deployment του Kubernetes, με μόνιμη διεκδίκηση αποθηκευτικού χώρου

(PersistentVolumeClaim, PVC) για τη διατήρηση των διανυσματικών αναπαραστάσεων μετά από επανεκκινήσεις.

Εικόνα 1: Διάγραμμα ροής του συστήματος Παραγωγής Επαυξημένης με Ανάκτηση (RAG).



2.3 Περιέκτες και Kubernetes

Οι περιέκτες (containers) προσφέρουν έναν ελαφρύ μηχανισμό συσκευασίας εφαρμογών μαζί με τις εξαρτήσεις τους, ώστε να εκτελούνται με την ίδια ακριβώς συμπεριφορά σε διαφορετικά περιβάλλοντα. Το Docker διέδωσε τη χρήση των περιεκτών και την αντίστοιχη ροή εργασίας, η οποία περιλαμβάνει το Dockerfile, την εικόνα περιέκτη (image), τον περιέκτη και το μητρώο εικόνων (registry). Το Kubernetes [5] είναι σήμερα ένα από τα πλέον διαδεδομένα συστήματα ενορχήστρωσης περιεκτών σε περιβάλλοντα παραγωγικής λειτουργίας.

2.3.1 Βασικά Αντικείμενα Kubernetes

Στην υλοποίηση χρησιμοποιούνται τα ακόλουθα αντικείμενα:

- **Pod:** η μικρότερη μονάδα ανάπτυξης · περιέχει έναν ή περισσότερους περιέκτες που μοιράζονται τον ίδιο χώρο ονομάτων δικτύου..
- **Deployment:** διαχειρίζεται τα αντίγραφα της εφαρμογής, τις κυλιόμενες ενημερώσεις (rolling updates) και την επαναφορά σε προηγούμενη έκδοση (rollback). Δηλώνεται με δηλωτικό τρόπο ο επιθυμητός αριθμός αντιγράφων.
- **Service:** σταθερό σημείο πρόσβασης, μέσω ονόματος DNS και διεύθυνσης ClusterIP, σε ένα σύνολο pods.
- **ConfigMap & Secret:** αντικείμενα που επιτρέπουν τον διαχωρισμό της διαμόρφωσης και των ευαίσθητων διαπιστευτηρίων πρόσβασης από τον κώδικα της εφαρμογής.
- **PersistentVolumeClaim (PVC):** αίτημα για μόνιμο αποθηκευτικό χώρο. Χρησιμοποιείται από το pod του ChromaDB, ώστε οι διανυσματικές αναπαραστάσεις να διατηρούνται μετά από επανεκκινήσεις.
- **DaemonSet:** εξασφαλίζει ότι εκτελείται ένα αντίγραφο του pod σε κάθε κατάλληλο κόμβο του υπολογιστικού συμπλέγματος
- **Init Container:** περιέκτης που εκτελείται μία φορά πριν από την εκκίνηση των κύριων περιεκτών του pod.

2.3.2 Μηχανισμός Οριζόντιας Αυτόματης Κλιμάκωσης (HPA)

Ο HPA [6] είναι ο ενσωματωμένος μηχανισμός του Kubernetes για αυτόματη οριζόντια κλιμάκωση. Παρακολουθεί μία ή περισσότερες μετρικές, όπως την αξιοποίηση της CPU, και προσαρμόζει τον αριθμό των αντιγράφων ενός Deployment, ώστε η μέση τιμή της επιλεγμένης μετρικής να προσεγγίζει την τιμή-στόχο. Η αξιοποίηση της CPU

υπολογίζεται ως ποσοστό του αντίστοιχου αιτήματος CPU που έχει δηλωθεί για τα pods.

Στην παρούσα υλοποίηση, η διαμόρφωση του HPA ορίζει στόχο μέσης αξιοποίησης CPU 70%, ελάχιστο αριθμό αντιγράφων minReplicas: 1 και μέγιστο αριθμό αντιγράφων maxReplicas: 5.

Ο HPA εκτελεί τον βρόχο ελέγχου του περιοδικά, με προεπιλεγμένη περίοδο συγχρονισμού περίπου 15 δευτερολέπτων. Για μετρικές πόρων, όπως η CPU, λαμβάνει τις απαραίτητες τιμές μέσω του API μετρικών πόρων, το οποίο στην παρούσα υλοποίηση εξυπηρετείται από τον metrics-server. Επιπλέον, εφόσον δεν έχει οριστεί διαφορετική πολιτική, εφαρμόζεται προεπιλεγμένο παράθυρο σταθεροποίησης 5 λεπτών για την κλιμάκωση προς τα κάτω, ώστε να αποφεύγεται η άμεση μείωση του αριθμού των αντιγράφων μετά από προσωρινές διακυμάνσεις της χρήσης CPU.

2.3.3 Αιτήματα και Όρια Πόρων

Κάθε περιέκτης στο Kubernetes δηλώνει την ελάχιστη ποσότητα πόρων που χρειάζεται (αιτήματα πόρων, requests) και τη μέγιστη ποσότητα πόρων που επιτρέπεται να χρησιμοποιήσει (όρια πόρων, limits). Τα αιτήματα πόρων χρησιμοποιούνται από τον χρονοπρογραμματιστή (scheduler) για την τοποθέτηση του pod σε κατάλληλο κόμβο και είναι απαραίτητα για τον υπολογισμό της αξιοποίησης CPU από τον HPA. Τα όρια πόρων μπορούν να οδηγήσουν σε περιορισμό της χρήσης CPU (throttling) ή σε τερματισμό του περιέκτη λόγω υπέρβασης της διαθέσιμης μνήμης (OOMKill), όταν ο περιέκτης τα υπερβεί.

2.4 Το Πρόβλημα της Ψυχρής Εκκίνησης στην Εξαγωγή Συμπερασμάτων από LLM

Όταν ο HPA δημιουργεί ένα νέο αντίγραφο ενός pod που εξυπηρετεί ένα LLM, το pod δεν είναι άμεσα έτοιμο να εξυπηρετήσει αιτήματα. Η διαδικασία της ψυχρής εκκίνησης περιλαμβάνει τα ακόλουθα στάδια:

- Χρονοπρογραμματισμός του pod: ανάθεση του pod σε έναν κόμβο, συνήθως σε χρόνο μικρότερο από ένα δευτερόλεπτο.
- Λήψη της εικόνας περιέκτη: αν η εικόνα δεν είναι ήδη διαθέσιμη στην τοπική προσωρινή μνήμη του κόμβου, λαμβάνεται από το μητρώο εικόνων. Για μια εικόνα μεγέθους περίπου 5 GB, η διαδικασία μπορεί να διαρκέσει από ένα έως τρία λεπτά, ανάλογα με το δίκτυο.
- Εκκίνηση του περιέκτη και αρχικοποίηση της υπηρεσίας Ollama: εκκίνηση της υπηρεσίας Ollama στον παράπλευρο περιέκτη (sidecar container), η οποία συνήθως διαρκεί μερικά δευτερόλεπτα.
- Διαθεσιμότητα του μοντέλου στον δίσκο: αν το μοντέλο δεν περιλαμβάνεται ήδη στην εικόνα περιέκτη, εκτελείται η εντολή ollama pull για τη λήψη του από το μητρώο μοντέλων του Ollama.
- Φόρτωση του μοντέλου στην κύρια μνήμη: το μοντέλο φορτώνεται από τον δίσκο μέσω λειτουργιών εισόδου/εξόδου με χαρτογράφηση μνήμης (memory-mapped I/O), δεσμεύεται μνήμη για τους τανυστές και προετοιμάζονται οι εσωτερικές δομές εκτέλεσης. Σε υποδομή αποκλειστικής χρήσης CPU, για μοντέλο μεγέθους 4,4 GB, το στάδιο αυτό μπορεί να απαιτήσει περίπου 30–90 δευτερόλεπτα.
- Αρχική προθέρμανση μέσω της πρώτης εξαγωγής συμπερασμάτων: η πρώτη πραγματική κλήση αρχικοποιεί την κρυφή μνήμη κλειδιών-τιμών (KV cache), ενεργοποιεί τις απαιτούμενες προσβάσεις στις σελίδες μνήμης και επιβαρύνεται με το κόστος της αρχικής προετοιμασίας.

Η συνολική διάρκεια της διαδικασίας μπορεί να φτάσει τα 5–7 λεπτά. Το κρίσιμο σημείο είναι ότι ένα pod σε κατάσταση Running στο Kubernetes δεν σημαίνει απαραίτητα ότι είναι έτοιμο να εξυπηρετήσει αιτήματα. Η αντιμετώπιση αυτής της διάκρισης, μέσω του συνδυασμού μηχανισμού προθέρμανσης και προσαρμοσμένων ανιχνευτών

ετοιμότητας που επιβεβαιώνουν τη διαθεσιμότητα του μοντέλου, αποτελεί έναν από τους πυλώνες της αρχιτεκτονικής που παρουσιάζεται στο Κεφάλαιο 4.

ΚΕΦΑΛΑΙΟ 3 Σχετική Έρευνα και Εργαλεία

Στο κεφάλαιο αυτό παρουσιάζονται τα κυριότερα εργαλεία και πλαίσια λογισμικού που έχουν αναπτυχθεί στους τομείς της εξυπηρέτησης μεγάλων γλωσσικών μοντέλων, του RAG και της ελαστικής κλιμάκωσης φόρτων εργασίας μηχανικής μάθησης. Στόχος δεν είναι μια εξαντλητική ανασκόπηση της βιβλιογραφίας, αλλά η τοποθέτηση των επιλογών της υλοποίησης σε ένα σαφές πλαίσιο.

3.1 Πλαίσια Εξυπηρέτησης Μεγάλων Γλωσσικών Μοντέλων

Ενδεικτικά, εξετάζονται τρία εργαλεία που χρησιμοποιούνται για την εξυπηρέτηση LLM σε περιβάλλοντα παραγωγικής λειτουργίας:

- `llama.cpp` [8]: μηχανή εξαγωγής συμπερασμάτων γραμμένη σε C++, με έμφαση στην απόδοση σε CPU και υποστήριξη κβαντισμένων μοντέλων, όπως μοντέλα σε μορφή GGUF και με κβάντιση Q4_K_M. Αποτελεί τη βάση πολλών παράγωγων εργαλείων.
- Ollama [9]: εργαλείο υψηλού επιπέδου για τη διαχείριση και εξυπηρέτηση γλωσσικών μοντέλων, το οποίο παρέχει διεπαφή προγραμματισμού REST, αυτοματοποιημένη διαχείριση μοντέλων μέσω της εντολής `ollama pull` και απλή ανάπτυξη μέσω μίας εικόνας περιέκτη. Επιλέχθηκε για την παρούσα εργασία λόγω της απλότητας της ενσωμάτωσής του στο Kubernetes.
- vLLM [7]: σύστημα εξυπηρέτησης μεγάλων γλωσσικών μοντέλων που αναπτύχθηκε στο UC Berkeley και εισήγαγε τις τεχνικές PagedAttention και συνεχούς ομαδοποίησης αιτημάτων (continuous batching). Οι τεχνικές αυτές βελτιώνουν τη διαχείριση της μνήμης

GPU και την ρυθμαπόδοση εξυπηρέτησης αιτημάτων. Δεν χρησιμοποιήθηκε στην παρούσα εργασία, επειδή στοχεύει κυρίως σε αναπτύξεις που αξιοποιούν GPU, ενώ το αντικείμενο της εργασίας αφορά την εξυπηρέτηση μοντέλων σε υποδομή αποκλειστικής χρήσης CPU

3.2 Πλαίσια Λογισμικού και Βάσεις Δεδομένων για RAG

Τα πλαίσια λογισμικού LangChain και LlamaIndex προσφέρουν έτοιμα δομικά στοιχεία για συστήματα RAG, όπως μηχανισμούς φόρτωσης εγγράφων, τεμαχισμού κειμένου, δημιουργίας διανυσματικών αναπαραστάσεων, αποθήκευσης διανυσμάτων και πρότυπα κειμένου εισόδου προς το μοντέλο. Στην παρούσα εργασία δεν χρησιμοποιήθηκαν τα συγκεκριμένα πλαίσια λογισμικού · ο κώδικας της διαδικασίας RAG (`app/rag.py`) γράφτηκε απευθείας, ώστε να υπάρχει πλήρης ορατότητα και έλεγχος στις επιμέρους λειτουργίες, όπως η ανίχνευση πρόθεσης, ο χειρισμός επακόλουθων ερωτημάτων και το φιλτράρισμα βάσει μεταδεδομένων. Η επιλογή αυτή καθιστά την υλοποίηση περισσότερο διαφανή για ακαδημαϊκή αξιολόγηση.

Στις διανυσματικές βάσεις δεδομένων, εκτός από το ChromaDB που τελικά επιλέχθηκε, υπάρχουν εμπορικές λύσεις, όπως το Pinecone, και άλλες λύσεις ανοικτού κώδικα, όπως τα Qdrant, Weaviate και Milvus. Το ChromaDB προτιμήθηκε επειδή μπορεί να φιλοξενηθεί τοπικά, διατίθεται ως απλή εικόνα περιέκτη Docker και ταιριάζει με τη φιλοσοφία της εργασίας, σύμφωνα με την οποία όλες οι υπηρεσίες εκτελούνται εντός του υπολογιστικού συμπλέγματος

3.3 Αυτόματη Κλιμάκωση σε Kubernetes

Πέρα από τον HPA που χρησιμοποιείται στην εργασία, υπάρχουν εναλλακτικές για πιο σύνθετα σενάρια:

- KEDA (Kubernetes-based Event-Driven Autoscaling) [11]: επιτρέπει την κλιμάκωση βάσει εξωτερικών πηγών συμβάντων, όπως ουρές

μηνυμάτων, θέματα Kafka και προσαρμοσμένες μετρικές, αντί να βασίζεται μόνο στη χρήση CPU ή μνήμης. Αξίζει να αναφερθεί, επειδή για φόρτους εργασίας LLM η πιο φυσική μετρική κλιμάκωσης μπορεί να είναι ο ρυθμός αιτημάτων ή το βάθος της ουράς αιτημάτων, και όχι η αξιοποίηση της CPU. Η αξιοποίηση του KEDA αναφέρεται ως μελλοντική εργασία στο Κεφάλαιο 7.

- Vertical Pod Autoscaler (VPA): προσαρμόζει τα αιτήματα και τα όρια πόρων αντί για τον αριθμό των αντιγράφων. Δεν είναι κατάλληλος για την εξυπηρέτηση LLM, όπου κάθε νέο αντίγραφο απαιτεί τη φόρτωση ολόκληρου του μοντέλου στην κύρια μνήμη.
- Cluster Autoscaler: προσθέτει ή αφαιρεί κόμβους από το υπολογιστικό σύμπλεγμα, όταν υπάρχουν pods που δεν μπορούν να προγραμματιστούν στους διαθέσιμους κόμβους. Η λειτουργία αυτή δεν αποτέλεσε αντικείμενο της παρούσας εργασίας, η οποία επικεντρώθηκε στην κλιμάκωση των αντιγράφων της εφαρμογής μέσω του HPA.

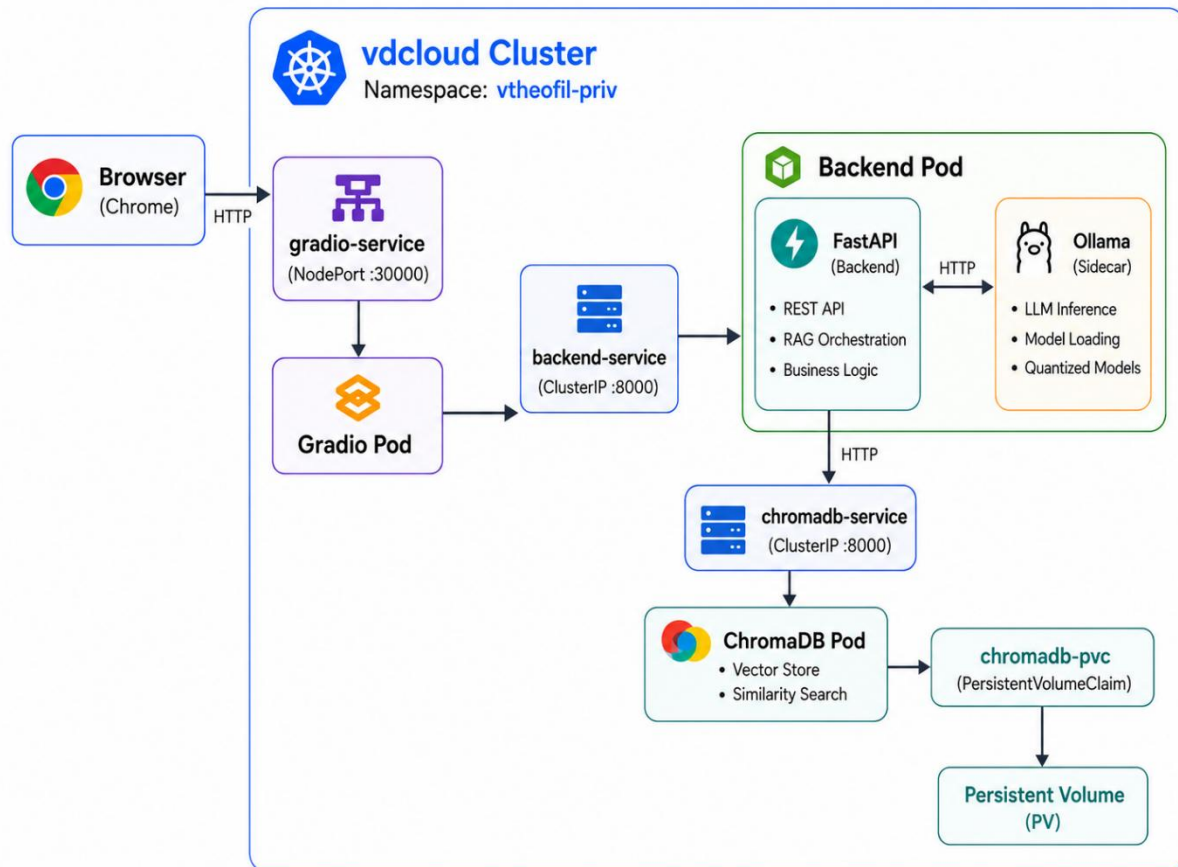
ΚΕΦΑΛΑΙΟ 4 Σχεδίαση του Συστήματος

4.1 Επισκόπηση Αρχιτεκτονικής

Το σύστημα αποτελείται από τέσσερα κύρια στοιχεία. Τα τρία πρώτα αναπτύσσονται ως ξεχωριστά Deployments στο υπολογιστικό σύμπλεγμα vndcloud, ενώ το Ollama εκτελείται ως παράπλευρος περιέκτης στο Backend:

- Gradio διεπαφή χρήστη — παρέχει γραφικό περιβάλλον σε φυλλομετρητή ιστού και αποστέλλει αιτήματα στο Backend.
- FastAPI Backend — εκθέτει τα τελικά σημεία REST (/chat, /chat/stream, /healthz, /readyz, /metrics) και ορχηστρώνει τη διαδικασία RAG.
- ChromaDB — διανυσματική βάση δεδομένων με μόνιμο αποθηκευτικό χώρο μέσω PVC.
- Ollama — παράπλευρος περιέκτης του Backend(sidcar), ο οποίος εξυπηρετεί το LLM (Phi-3 Mini ή Mistral 7B) τοπικά, στη διεύθυνση localhost:11434.

Εικόνα 2: Αρχιτεκτονική του συστήματος.



4.2 Επιλογή Γνωσιακής Βάσης — Atlas Systems

Κατά την αρχική φάση σχεδιασμού αξιολογήθηκαν διαφορετικά είδη γνωσιακών βάσεων. Επιλέχθηκε τελικά μια δομημένη εταιρική γνωσιακή βάση, καθώς επιτρέπει αντικειμενική αξιολόγηση της διαδικασίας ανάκτησης πληροφοριών και μειώνει την εξάρτηση από σύνθετες δυνατότητες συλλογισμού του γλωσσικού μοντέλου. Δεδομένου ότι η εργασία επικεντρώνεται στην υποδομή και όχι στη νοημοσύνη του μοντέλου, το πεδίο εφαρμογής πρέπει να ευνοεί την ανάκτηση και παρουσίαση πραγματολογικών πληροφοριών, όπου τα μικρότερα μοντέλα που εκτελούνται σε CPU μπορούν να ανταποκριθούν αξιόπιστα, και όχι δημιουργικές εργασίες ή εργασίες συλλογισμού πολλαπλών βημάτων.

Η βάση «Atlas Systems» αποτελεί συνθετική εταιρική γνωσιακή βάση που καλύπτει οκτώ κατηγορίες δεδομένων: υπαλλήλους, έργα, τμήματα, εταιρικές πολιτικές, συχνές ερωτήσεις τεχνικής υποστήριξης, συναντήσεις και εκδηλώσεις, μηνιαία πλάνα εργασιών και αγγελίες θέσεων εργασίας. Επιπλέον, περιλαμβάνει έντεκα σελίδες εταιρικής παρουσίασης, όπως εταιρική επισκόπηση, αποστολή, ηγεσία, γραφεία, πελάτες και σημαντικά ορόσημα, οι οποίες τεκμηριώνουν τη φανταστική εταιρεία ως συνεκτικό σύνολο.

4.3 Επιλογή Συνόλου Τεχνολογιών

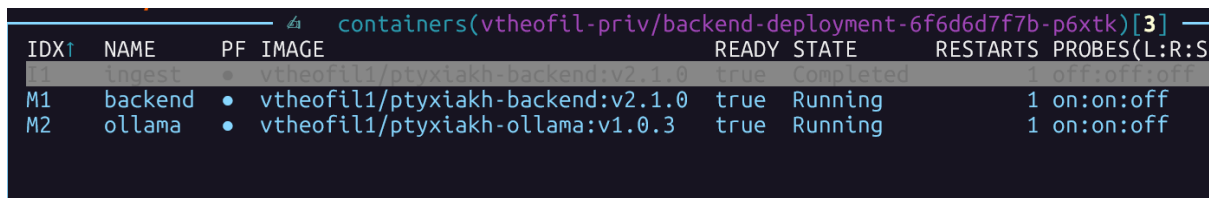
Οι επιλογές των τεχνολογιών έγιναν με γνώμονα την παραγωγικότητα ανάπτυξης, την καταλληλότητα για εκτέλεση σε υποδομές αποκλειστικής χρήσης CPU και τη συμβατότητα με τις αρχές νεφοεγγενούς ανάπτυξης που υιοθετεί το Kubernetes:

- FastAPI για το backend: σύγχρονο πλαίσιο ανάπτυξης εφαρμογών Python με ασύγχρονη υποστήριξη και αυτόματη τεκμηρίωση OpenAPI.
- ChromaDB ως διανυσματική βάση δεδομένων: αυτοφιλοξενούμενη λύση, με υποστήριξη φιλτραρίσματος βάσει μεταδεδομένων και μόνιμη αποθήκευση μέσω PVC.
- Ollama για την εξυπηρέτηση του LLM: παρέχει διεπαφή REST, ενσωματωμένη διαχείριση μοντέλων και δυνατότητα προενσωμάτωσης του μοντέλου σε εικόνα περιέκτη Docker.
- multilingual-e5-small για τις διανυσματικές αναπαραστάσεις: κατάλληλο για εξαγωγή συμπερασμάτων σε CPU και με υποστήριξη πολλών γλωσσών.
- Gradio για τη διεπαφή χρήστη: επιτρέπει τη γρήγορη δημιουργία ενός περιβάλλοντος συνομιλίας χωρίς την ανάγκη ανάπτυξης ξεχωριστού κώδικα διεπαφής.

4.4 Διάταξη Παράπλευρου Περιέκτη (FastAPI + Ollama)

Το pod του Backend περιέχει δύο περιέκτες σε διάταξη παράπλευρου περιέκτη (sidecar): τον περιέκτη FastAPI και τον περιέκτη Ollama. Οι δύο περιέκτες μοιράζονται τον ίδιο χώρο ονομάτων δικτύου, επομένως η FastAPI επικοινωνεί με το Ollama στη διεύθυνση localhost:11434, χωρίς να μεσολαβεί δικτυακή επικοινωνία μεταξύ διαφορετικών pods. Η επιλογή αυτή μειώνει τη λανθάνουσα καθυστέρηση των κλήσεων και απλοποιεί τη διαχείριση της εφαρμογής, καθώς οι δύο περιέκτες κλιμακώνονται μαζί ως μία ενιαία μονάδα.

Εικόνα 3: Διάταξη παράπλευρου περιέκτη FastAPI και Ollama στο pod του Backend.



IDX↑	NAME	PF	IMAGE	READY	STATE	RESTARTS	PROBES(L:R:S)
1	ingest	•	vtheofil/ptyxiakh-backend:v2.1.0	true	Completed	1	off:off:off
M1	backend	•	vtheofil1/ptyxiakh-backend:v2.1.0	true	Running	1	on:on:off
M2	ollama	•	vtheofil1/ptyxiakh-ollama:v1.0.3	true	Running	1	on:on:off

4.5 Περιέκτης Αρχικοποίησης (Init Container) για Εισαγωγή Δεδομένων (ingestion)

Πριν από την εκκίνηση των κύριων περιεκτών του pod του Backend, ένας περιέκτης αρχικοποίησης αναμένει έως ότου γίνει διαθέσιμο το ChromaDB, μέσω περιοδικού ελέγχου του endpoint ετοιμότητας. Στη συνέχεια, εκτελεί το σενάριο app/ingest.py, το οποίο διαβάζει τα έγγραφα Markdown, τα τεμαχίζει σε μικρότερα τμήματα κειμένου, τα μετατρέπει σε διανυσματικές αναπαραστάσεις και τα αποθηκεύει στο ChromaDB.

Με τον τρόπο αυτό, ο κύριος περιέκτης του Backend εκκινεί μόνο αφού η διανυσματική βάση δεδομένων έχει προετοιμαστεί και περιέχει τα απαραίτητα δεδομένα της γνωσιακής βάσης.

4.6 Μοτίβο Προδημιουργημένης Εικόνας (Pre-Baked Image)

Αντί να εκτελείται η εντολή ollama pull κατά τον χρόνο εκτέλεσης για κάθε νέο pod, το μοντέλο ενσωματώνεται στην ίδια την εικόνα Docker

κατά τη φάση δημιουργίας της εικόνας (docker build). Η εικόνα αυξάνεται σημαντικά σε μέγεθος, από περίπου 300 MB για την υπηρεσία Ollama χωρίς μοντέλο σε αρκετά GB, ανάλογα με το ενσωματωμένο μοντέλο.

Με τον τρόπο αυτό εξαλείφεται η λήψη του μοντέλου κατά την εκκίνηση ενός νέου pod. Ωστόσο, εξακολουθούν να απαιτούνται η λήψη της ίδιας της εικόνας περιέκτη, όταν αυτή δεν είναι ήδη διαθέσιμη στον κόμβο, καθώς και η φόρτωση του μοντέλου στην κύρια μνήμη. Στην παρούσα υλοποίηση υπάρχουν δύο προδημιουργημένες εικόνες:

- vtheofil1/ptyxiakh-ollama:v1.0.3 — περιέχει το Phi-3 Mini.
- vtheofil1/ptyxiakh-ollama-mistral:v1.0.0 — περιέχει το Mistral 7B.

4.7 Μοτίβο Προθέρμανσης Κόμβων μέσω DaemonSet (DaemonSet Pre-Warm)

Το μοτίβο προθέρμανσης μέσω DaemonSet εξασφαλίζει ότι οι προδημιουργημένες εικόνες περιεκτών είναι διαθέσιμες στην τοπική προσωρινή μνήμη κάθε κατάλληλου κόμβου του υπολογιστικού συμπλέγματος. Ένας DaemonSet διατηρεί ένα pod σε κάθε κόμβο, το οποίο δεν εξυπηρετεί αιτήματα, αλλά χρησιμοποιεί την προδημιουργημένη εικόνα περιέκτη. Με τον τρόπο αυτό, το kubelet λαμβάνει την εικόνα στον αντίστοιχο κόμβο. Επιπλέον, όσο ο περιέκτης του DaemonSet εκτελείται, η εικόνα θεωρείται ενεργά χρησιμοποιούμενη και δεν επιλέγεται από τον μηχανισμό συλλογής αχρήστων εικόνων (image garbage collection).

Αξιολόγηση αντισταθμίσεων:

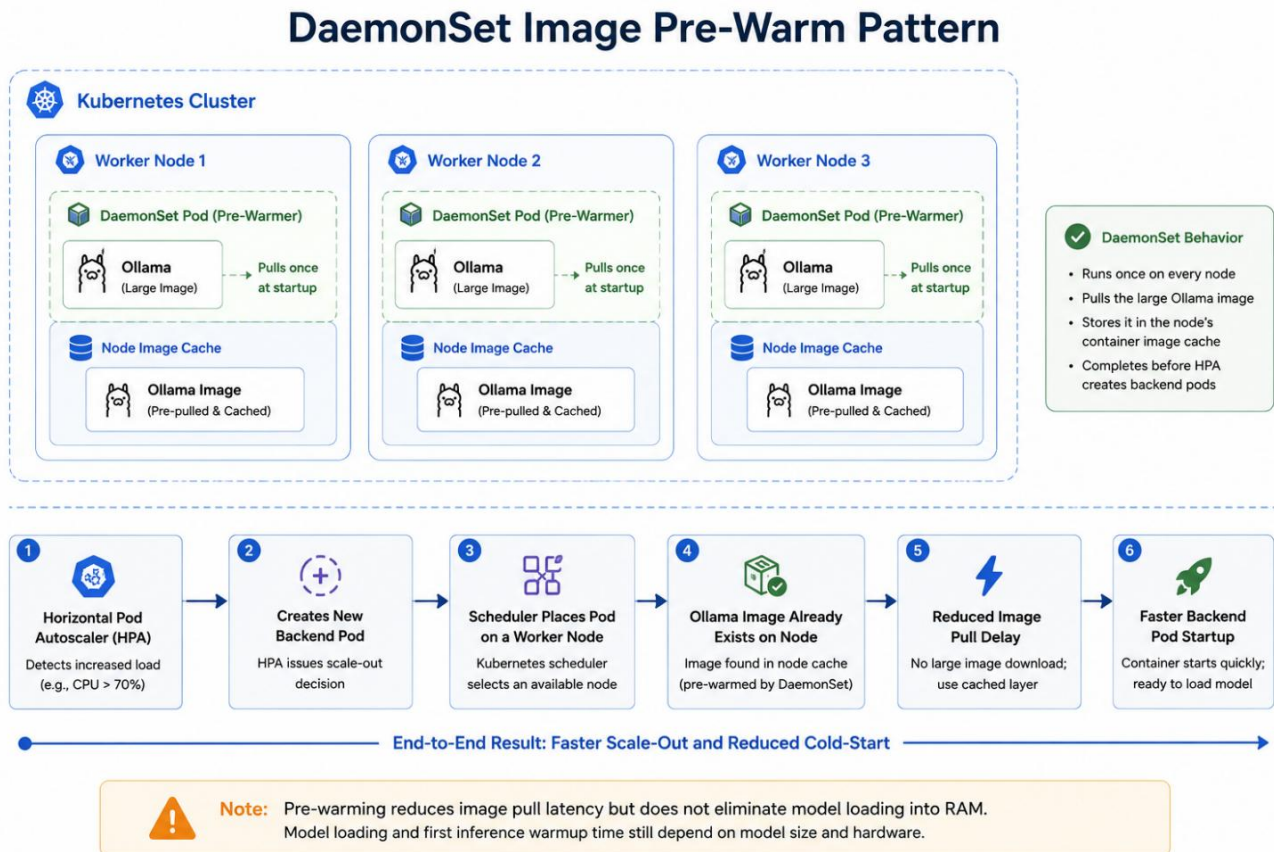
- (+) Μειώνει την καθυστέρηση λήψης της εικόνας (image-pull delay), όταν ο HPA δημιουργεί νέο pod σε κόμβο που δεν έχει προηγουμένως εξυπηρετήσει φόρτο εργασίας LLM.

- (–) Καταναλώνει αποθηκευτικό χώρο στους δίσκους όλων των κόμβων, περίπου 5 GB για κάθε εικόνα περιέκτη.
- (–) Δημιουργεί αιχμή δικτυακής κίνησης κατά την αρχική εφαρμογή του DaemonSet ή κατά την ενημέρωση της εικόνας περιέκτη.

Εικόνα 4: Μηχανισμός προθέρμανσης κόμβων μέσω DaemonSet.

ollama-image-preloader-4nkzt	0m	3Mi
ollama-image-preloader-5h6hp	0m	0Mi
ollama-image-preloader-5j6t5	0m	0Mi
ollama-image-preloader-c8dzd	0m	1Mi
ollama-image-preloader-cp5f2	0m	1Mi
ollama-image-preloader-f2x5g	0m	0Mi
ollama-image-preloader-ghlt6	0m	1Mi
ollama-image-preloader-gmyn5	0m	0Mi
ollama-image-preloader-h7cjt	0m	0Mi
ollama-image-preloader-j4j6v	0m	1Mi
ollama-image-preloader-jxn8t	0m	1Mi
ollama-image-preloader-nfbfp	0m	0Mi
ollama-image-preloader-pxnc9	0m	1Mi
ollama-image-preloader-qh4qw	0m	3Mi
ollama-image-preloader-sd4cr	0m	1Mi
ollama-image-preloader-sxsrl	0m	0Mi
ollama-image-preloader-vmrm9	0m	3Mi
ollama-image-preloader-w7j29	0m	0Mi
ollama-image-preloader-w7jfk	0m	1Mi
ollama-image-preloader-wkk9f	0m	3Mi

Εικόνα 5: Ανάπτυξη του DaemonSet προθέρμανσης κόμβων.



4.8 Μηχανισμός postStart και Προσαρμοσμένος Ανιχνευτής Ετοιμότητας (Custom Readiness Probe)

Ακόμη και όταν η εικόνα περιέκτη είναι ήδη διαθέσιμη στην τοπική προσωρινή μνήμη του κόμβου, το LLM πρέπει να φορτωθεί στην κύρια μνήμη (RAM) πριν το pod μπορέσει να εξυπηρετήσει αιτήματα. Για τον σκοπό αυτό συνεργάζονται δύο μηχανισμοί:

- Μηχανισμός postStart στον περιέκτη Ollama: μετά την εκκίνηση της υπηρεσίας Ollama, εκτελείται ένα αίτημα προθέρμανσης του μοντέλου, για παράδειγμα `ollama run mistral "ok"`. Με τον τρόπο αυτό το μοντέλο φορτώνεται στην κύρια μνήμη και ολοκληρώνεται η αρχική προθέρμανση πριν το pod αρχίσει να εξυπηρετεί πραγματικά αιτήματα.

• Ανιχνευτής ετοιμότητας `readinessProbe` με εντολή εκτέλεσης (`ollama list | grep -q <model>`): το pod δεν προστίθεται στα διαθέσιμα τελικά σημεία του Service έως ότου το συγκεκριμένο μοντέλο εμφανιστεί στη λίστα του Ollama. Ο έλεγχος αυτός επιβεβαιώνει ότι το μοντέλο είναι διαθέσιμο στο περιβάλλον του Ollama, χωρίς όμως να εγγυάται από μόνος του ότι παραμένει ενεργά φορτωμένο στην κύρια μνήμη. Σε συνδυασμό με τον μηχανισμό `postStart` και τη ρύθμιση `OLLAMA_KEEP_ALIVE`, μειώνει σημαντικά την πιθανότητα ένα νέο pod να λάβει αιτήματα πριν ολοκληρωθεί η απαραίτητη προετοιμασία του μοντέλου.

Εικόνα 6: Ανιχνευτής ετοιμότητας (`readinessProbe`) του περιέκτη Ollama.

```
readinessProbe:
  exec:
    command:
      - sh
      - -c
      - ollama list | grep -q phi3
```

Εικόνα 7: Μηχανισμός `postStart` για την προθέρμανση του μοντέλου.

```
lifecycle:
  postStart:
    exec:
      command:
        - sh
        - -c
        - |
          # 1. Wait for the Ollama daemon to be ready
          until ollama list >/dev/null 2>&1; do sleep 2; done
          # 2. Pull the model only if missing (safety net – baked-in
          #    image already has it)
          ollama list | grep -q phi3 || ollama pull phi3:mini
          ollama run phi3:mini "ok" >/dev/null 2>&1 || true
```

4.9 Ρύθμιση OLLAMA_KEEP_ALIVE — Αντιστάθμιση μεταξύ Προθερμασμένων Αντιγράφων (Warm Pool) και Πίεσης Μνήμης

Ο Ollama, με την προεπιλεγμένη του ρύθμιση, μπορεί να αποφορτώνει το μοντέλο από την κύρια μνήμη μετά από μια περίοδο αδράνειας. Σε ένα σύστημα που βασίζεται σε προθερμασμένα αντίγραφα, η συμπεριφορά αυτή δεν είναι επιθυμητή, επειδή ένα pod που εμφανίζεται διαθέσιμο μπορεί να χρειαστεί ξανά χρόνο για να φορτώσει το μοντέλο στην κύρια μνήμη κατά το επόμενο αίτημα.

Για τον λόγο αυτό χρησιμοποιήθηκε η ρύθμιση `OLLAMA_KEEP_ALIVE=24h`, ώστε τα αντίγραφα να διατηρούν το μοντέλο φορτωμένο για μεγάλο χρονικό διάστημα και να μειώνεται η καθυστέρηση του πρώτου αιτήματος μετά από περίοδο αδράνειας. Η επιλογή αυτή βελτιώνει τη συμπεριφορά του συνόλου προθερμασμένων αντιγράφων (warm pool), αλλά αυξάνει τη μόνιμη κατανάλωση κύριας μνήμης ανά pod.

Στα πειράματα παρατηρήθηκε ότι ο περιέκτης Ollama μπορούσε να διατηρεί αρκετά GiB δεσμευμένης κύριας μνήμης ακόμη και όταν δεν εξυπηρετούσε ενεργά αιτήματα. Σε υψηλότερο ή παρατεταμένο φόρτο εργασίας, η αυξημένη χρήση μνήμης οδήγησε σε πίεση μνήμης σε επίπεδο κόμβου και, σε ορισμένες περιπτώσεις, σε απομάκρυνση ή τερματισμό pods.

Επομένως, η `OLLAMA_KEEP_ALIVE` δεν αποτελεί απλώς ρύθμιση απόδοσης, αλλά βασική αντιστάθμιση ανάμεσα στη γρήγορη απόκριση των προθερμασμένων pods και στη σταθερότητα της μνήμης του υπολογιστικού συμπλέγματος. Για τα πειράματα με 3–10 ταυτόχρονους χρήστες, η τιμή 24h ήταν χρήσιμη, επειδή διατηρούσε τα αντίγραφα έτοιμα για εξαγωγή συμπερασμάτων. Ωστόσο, σε περιβάλλον παραγωγικής λειτουργίας με συνεχές υψηλό φορτίο θα απαιτούνταν προσεκτικότερη πολιτική, όπως μικρότερος χρόνος διατήρησης του μοντέλου στη μνήμη, περιορισμός της ουράς αιτημάτων, υψηλότερα αιτήματα μνήμης ή περιοδική επανεκκίνηση των pods.

4.10 Διαμόρφωση HPA

Η διαμόρφωση του HPA για το Deployment της υπηρεσίας υποστήριξης είναι η ακόλουθη:

- Στόχος μετρικής: μέση αξιοποίηση CPU ίση με 70%.
- minReplicas: 1 — το σύστημα ξεκινά από ένα μόνο pod, ώστε η διαδικασία ελαστικής κλιμάκωσης να είναι παρατηρήσιμη κατά την εκτέλεση των πειραμάτων.
- maxReplicas: 5 — ένα συντηρητικά επιλεγμένο ανώτατο όριο. Επειδή το vndcloud αποτελεί κοινόχρηστο υπολογιστικό σύμπλεγμα με πολλούς χρήστες και κάθε pod της υπηρεσίας υποστήριξης έχει σημαντικές απαιτήσεις πόρων, η τιμή αυτή επιλέχθηκε ώστε να επιτρέπει την παρατηρήσιμη κλιμάκωση κατά τα πειράματα, χωρίς να επιβαρύνει υπερβολικά τους υπόλοιπους χρήστες του συμπλέγματος.

Κατά την ενεργή εξυπηρέτηση αιτημάτων, ένα pod μπορεί να χρησιμοποιεί περίπου 5 GiB κύριας μνήμης για το Mistral 7B, περίπου 3 GiB για το Phi-3 Mini και έως 4 εικονικούς πυρήνες CPU, σύμφωνα με τα δηλωμένα όρια πόρων. Η ακριβής συνολική χωρητικότητα του vndcloud δεν ήταν διαθέσιμη στον χρήστη του συγκεκριμένου χώρου ονομάτων. Επομένως, η τιμή maxReplicas: 5 δεν παρουσιάζεται ως μαθηματικά υπολογισμένο όριο, αλλά ως συντηρητική λειτουργική επιλογή για τον πειραματικό φόρτο εργασίας.

4.11 Πολυμοντελική Αρχιτεκτονική για Συγκριτική Μελέτη

Για να επιτευχθεί άμεση και δίκαιη σύγκριση μεταξύ Phi-3 Mini και Mistral 7B, σχεδιάστηκε μια πολυμοντελική αρχιτεκτονική. Και τα δύο μοντέλα είναι ενσωματωμένα σε ξεχωριστές προδημιουργημένες εικόνες περιεκτών, οι οποίες είναι διαθέσιμες στο Docker Hub. Η εναλλαγή μεταξύ μοντέλων γίνεται με δύο αλλαγές: στο πεδίο image: του YAML του Deployment και στη σταθερά LLM_MODEL του app/rag.py.

Το DaemonSet προθέρμανσης κόμβων διατηρεί και τις δύο εικόνες διαθέσιμες στην τοπική προσωρινή μνήμη κάθε κόμβου. Έτσι, η

σύγκριση δεν επηρεάζεται από την καθυστέρηση λήψης της εικόνας κατά τον χρόνο εκτέλεσης, καθώς κατά την αλλαγή μοντέλου η νέα εικόνα είναι ήδη διαθέσιμη σε όλους τους κόμβους. Η μεθοδολογική αυτή πρόβλεψη περιορίζει έναν σημαντικό συγχυτικό παράγοντα: οι διαφορές στους μετρούμενους χρόνους απόκρισης δεν επηρεάζονται από άνιση κατάσταση της προσωρινής μνήμης εικόνων του υπολογιστικού συμπλέγματος, αλλά αποδίδονται κυρίως στη συμπεριφορά των δύο μοντέλων.

ΚΕΦΑΛΑΙΟ 5 Υλοποίηση

Στο κεφάλαιο αυτό περιγράφεται η τεχνική υλοποίηση του συστήματος.

5.1 Οργάνωση του Project

Ο φάκελος του έργου έχει την ακόλουθη δομή:

- `app/` — υπηρεσία FastAPI, διαδικασία RAG (`rag.py`), σενάριο εισαγωγής δεδομένων (`ingest.py`) και μοντέλα Pydantic στο `main.py`.
- `ui/` — διεπαφή συνομιλίας Gradio (`gradio_app.py`).
- `δεδομένα/` — αρχεία Markdown της γνωσιακής βάσης Atlas Systems, οργανωμένα σε υποφακέλους ανά κατηγορία.
- `k8s/` — αρχεία δηλωτικής διαμόρφωσης Kubernetes (`deployment.yaml`, `service.yaml`, `hpa.yaml`, `configmap.yaml`, `secret.yaml`, `preload-daemonset.yaml`, `locust-deployment.yaml`, `locust-configmap.yaml`).
- `locust/` — ορισμός δοκιμών φόρτου Locust (`locustfile.py`).
- `scripts/` — βοηθητικά σενάρια, συμπεριλαμβανομένου του `generate_atlas_data.py`.
- `Dockerfile`, `Dockerfile.gradio`, `Dockerfile.ollama`, `Dockerfile.ollama.mistral` — τα αρχεία δημιουργίας των τεσσάρων εικόνων Docker του έργου.
- `Makefile` — ροή εργασίας για δημιουργία, δημοσίευση, ανάπτυξη και επαναφορά εκδόσεων.

5.2 Παραγωγή του Συνθετικού Συνόλου Δεδομένων (Synthetic Dataset) Atlas Systems

Τα έγγραφα της βάσης γνώσης παράγονται προγραμματιστικά από το `scripts/generate_atlas_data.py`, χρησιμοποιώντας τη βιβλιοθήκη `Faker`, με σταθερό σπόρο τυχαιότητας (seed) 42 για επαναληψιμότητα. Η τρέχουσα διάταξη παράγει:

- 150 αρχεία υπαλλήλων (δεδομένα/`employees/`)
- 50 αρχεία έργων (δεδομένα/`projects/`)
- 10 αρχεία τμημάτων (δεδομένα/`departments/`)
- 10 αρχεία εταιρικών πολιτικών (δεδομένα/`policies/`)
- 10 αρχεία συχνών ερωτήσεων τεχνικής υποστήριξης (δεδομένα/`it_faq/`)
- 150 αρχεία συναντήσεων (δεδομένα/`meetings/`)
- 36 αρχεία μηνιαίων πλάνων (δεδομένα/`monthly_plans/`, τρία έτη × δώδεκα μήνες)
- 20 αρχεία αγγελιών θέσεων εργασίας (δεδομένα/`job_postings/`)
- 11 αρχεία εταιρικής παρουσίασης (δεδομένα/`about/`), όπως `company_overview`, `mission_vision`, `leadership`, `offices`, `services`, `client_portfolio`, `technology_stack`, `partnerships`, `hiring_process`, `employee_benefits` και `company_milestones`.

Σύνολο: 447 αρχεία Markdown. Κάθε αρχείο περιλαμβάνει μεταδεδομένα YAML στην επικεφαλίδα του (front matter), όπως `type`, `title`, `name`, `department` και `project`, τα οποία χρησιμοποιούνται για φιλτράρισμα βάσει μεταδεδομένων κατά την ανάκτηση πληροφοριών. Σημαντικό στοιχείο για την ποιότητα του συνόλου δεδομένων είναι το λεξικό (dictionary) `LEADERSHIP_ROLE`, το οποίο εξασφαλίζει ότι κάθε τμήμα έχει ακριβώς έναν επικεφαλής. Διαφορετικά, οι σχετικές πληροφορίες μπορεί να είναι αντιφατικές μέσα στη βάση γνώσης και το LLM να παράγει εσφαλμένες απαντήσεις.

5.3 FastAPI Backend (app/main.py)

Το Backend εκθέτει τα ακόλουθα τελικά σημεία:

- POST /chat — σημείο συνομιλίας χωρίς ροή απάντησης. Λαμβάνει μήνυμα και προαιρετικό ιστορικό, και επιστρέφει την πλήρη απάντηση μαζί με τις πηγές.
- POST /chat/stream — σημείο συνομιλίας με ροή απάντησης. Επιστρέφει την απάντηση ανά διακριτό τμήμα κειμένου ως ροή text/plain.
- GET /healthz — έλεγχος λειτουργικότητας (liveness probe) του Kubernetes. Επιβεβαιώνει ότι η εφαρμογή FastAPI ανταποκρίνεται.
- GET /readyz — ανιχνευτής ετοιμότητας του Kubernetes. Ελέγχει τη συνδεσιμότητα προς το ChromaDB και το Ollama. Επιστρέφει κωδικό 503 αν κάποια εξάρτηση δεν είναι διαθέσιμη.
- GET /metrics — εκθέτει βασικές μετρικές, όπως πλήθος αιτημάτων, συνολική καθυστέρηση και σφάλματα.

Κατά την εκκίνηση της εφαρμογής FastAPI φορτώνεται προληπτικά το μοντέλο διανυσματικών αναπαραστάσεων, ώστε το πρώτο αίτημα να μην επιβαρύνεται με το κόστος φόρτωσης του μοντέλου.

5.4 Διαδικασία RAG (app/rag.py)

Το app/rag.py υλοποιεί τη βασική λογική του RAG. Οι σημαντικότερες ρυθμίσεις είναι:

- COLLECTION = "enterprise_docs" — όνομα της συλλογής του ChromaDB.
- TOP_K = 3 — αριθμός ανακτημένων τμημάτων κειμένου (μειώθηκε από 5 σε 3, ώστε τα μικρότερα μοντέλα να μην συγχέονται).

- `TIMEOUT = 1200.0` δευτερόλεπτα — μεγάλο όριο για να καλύπτει σενάρια ψυχρής εκκίνησης, όπου το Ollama πρέπει να φορτώσει το μοντέλο στην κύρια μνήμη.
- `HISTORY_RETRIEVAL_TURNS = 1`, `HISTORY_PROMPT_TURNS = 2` — ελέγχουν πόσοι προηγούμενοι γύροι του διαλόγου χρησιμοποιούνται κατά την ανάκτηση και στο κείμενο εισόδου προς το μοντέλο, αντίστοιχα.

Η διαδικασία εκτελεί:

- Δημιουργία διανυσματικής αναπαράστασης του ερωτήματος με το `multilingual-e5-small`, με το πρόθεμα `"query:"`, όπως απαιτεί η σύμβαση του E5.
- Ανίχνευση πρόθεσης μέσω προτύπων λέξεων-κλειδιών — για παράδειγμα, ερωτήσεις σχετικές με έργα ενεργοποιούν το φίλτρο `where={"type": "project"}` στο ChromaDB, ώστε να αποκλείονται τμήματα άλλων τύπων, όπως `monthly_plan`, που απλώς αναφέρουν τυχαία το όνομα του έργου.
- Ανίχνευση επακόλουθων ερωτημάτων («tell me more», «what about it») μέσω προτύπων κανονικών εκφράσεων. Σε αυτή την περίπτωση, ως βάση για την ανάκτηση συνδυάζεται το προηγούμενο μήνυμα του χρήστη με την τρέχουσα είσοδο.
- Δημιουργία κειμένου εισόδου προς το μοντέλο με οδηγίες συστήματος, ανακτημένο πλαίσιο και ιστορικό διαλόγου.
- Κλήση του Ollama μέσω `httpx`, με ροή απάντησης ή πλήρη απάντηση.

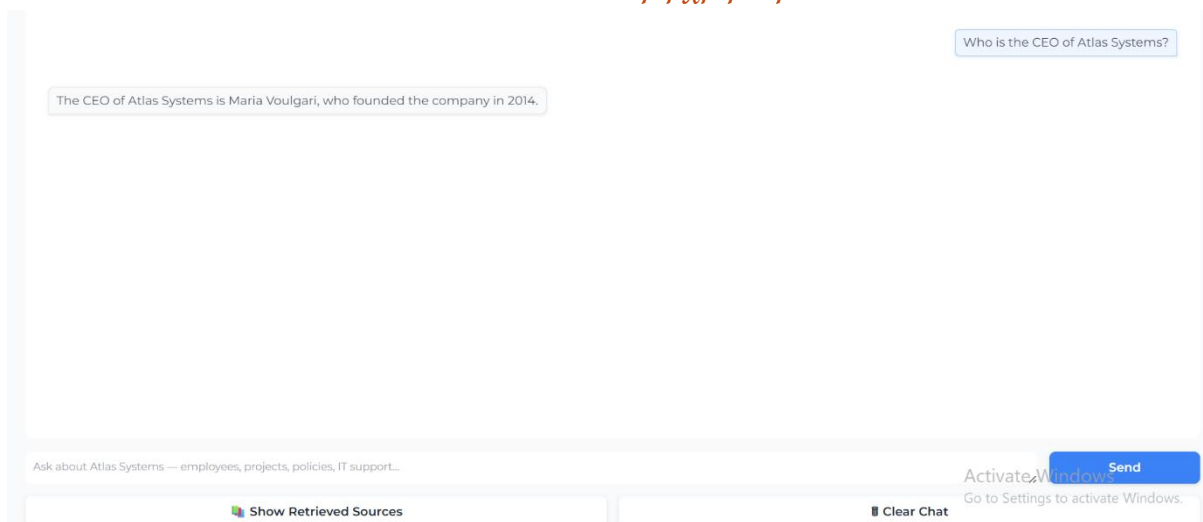
5.5 Διεπαφή Χρήστη Gradio (`ui/gradio_app.py`)

Η διεπαφή χρήστη είναι ένα περιβάλλον συνομιλίας Gradio. Υλοποιεί μια διαδικασία υποβολής δύο βημάτων, ώστε το μήνυμα του χρήστη να εμφανίζεται άμεσα, πριν φτάσει η απάντηση του LLM, η οποία μπορεί να καθυστερήσει για αρκετά δευτερόλεπτα. Η υλοποίηση ακολουθεί ένα

συνηθισμένο πρότυπο: η συνάρτηση `put_message_in_chatbot` εμφανίζει άμεσα την είσοδο του χρήστη και η συνάρτηση `chat(history)` χειρίζεται την κλήση προς το Backend και επιστρέφει το ενημερωμένο ιστορικό συνομιλίας.

Κατά τη διάρκεια της ανάπτυξης εντοπίστηκε και διορθώθηκε ένα σφάλμα στη μεταφορά του ιστορικού από το Gradio προς το Backend, όπου το ιστορικό μετατρεπόταν δύο φορές σε διαφορετικές μορφές δεδομένων, με αποτέλεσμα να φτάνει κενό στο Backend. Η διόρθωση εξασφάλισε ότι η συνομιλία πολλαπλών γύρων λειτουργεί όπως αναμένεται.

Εικόνα 9: Διεπαφή χρήστη Gradio.



Εικόνα 8: Ανακτημένα τμήματα κειμένου από τη διανυσματική βάση δεδομένων.

← Back to Chat

Retrieved chunks (top-k from ChromaDB)

1. About / Atlas Systems Leadership Team

data/about/leadership.md

cosine distance: 0.1046

```
# Leadership Team
Atlas Systems is led by **CEO Maria Voulgari**, supported by an executive team representing the company's primary functions. Department directors report directly to the CEO.
## Executive Office
- **Maria Voulgari** - Chief Executive Officer (CEO). Founder of Atlas Systems (2014).
## Direct Reports to the CEO
Each department is headed by its director or equivalent C-level executive, all of whom report to the CEO:
```

5.6 Δημιουργία Εικόνων Περιεκτών (Containerization)

Υπάρχουν τέσσερις εικόνες Docker στο έργο:

- **Dockerfile (Backend): πολυσταδιακή διαδικασία δημιουργίας (multi-stage build)** με Python 3.11-slim. Στο πρώτο στάδιο εγκαθίστανται οι εξαρτήσεις Python, συμπεριλαμβανομένου του PyTorch για CPU. Στο δεύτερο στάδιο αντιγράφονται τα πακέτα στην **ελαφριά εικόνα εκτέλεσης**, προφορτώνεται το μοντέλο διανυσματικών αναπαραστάσεων κατά τη φάση δημιουργίας της εικόνας και δηλώνεται χρήστης χωρίς δικαιώματα διαχειριστή (appuser).
- **Dockerfile.gradio:** μικρότερη εικόνα με Gradio και httpx. Οι εκδόσεις των πακέτων (gradio 4.44.1, gradio_client 1.3.0, fastapi 0.115.0, starlette 0.38.6, jinja2 3.1.4, huggingface_hub<0.26) είναι **δεσμευμένες σε συγκεκριμένες τιμές**, επειδή νεότεροι συνδυασμοί εκδόσεων προκαλούν σφάλματα στο Gradio 4.44.1.
- **Dockerfile.ollama:** βασίζεται στην επίσημη εικόνα ollama/ollama:latest και εκτελεί ollama pull phi3:mini κατά τη φάση δημιουργίας της εικόνας, ώστε το μοντέλο να ενσωματώνεται στην τελική εικόνα.
- **Dockerfile.ollama.mistral:** το αντίστοιχο αρχείο για το Mistral 7B.

5.7 Αρχεία Δηλωτικής Διαμόρφωσης Kubernetes (Manifests)

Τα αρχεία δηλωτικής διαμόρφωσης βρίσκονται στον φάκελο `k8s/` και ομαδοποιούνται ως εξής:

- `configmap.yaml` — μη ευαίσθητη διαμόρφωση (`OLLAMA_URL`, `CHROMA_HOST`, `CHROMA_PORT`, `BACKEND_URL`).
- `secret.yaml` — θέση για ευαίσθητα διαπιστευτήρια πρόσβασης· δεν περιέχει πραγματικές τιμές.
- `deployment.yaml` — τρία Deployments σε ένα αρχείο: `chromadb-deployment` (με PVC), `backend-deployment` (με περιέκτη αρχικοποίησης για εισαγωγή δεδομένων, περιέκτη FastAPI και παράπλευρο περιέκτη Ollama με μηχανισμό `postStart` και προσαρμοσμένο ανιχνευτή ετοιμότητας) και `gradio-deployment`.
- `service.yaml` — `chromadb-service`, `backend-service` (ClusterIP) και `gradio-service` (NodePort 30000).
- `hpa.yaml` — `backend-hpa` με στόχο μέσης αξιοποίησης CPU 70%, `minReplicas`: 1 και `maxReplicas`: 5.
- `preload-daemonset.yaml` — το μοτίβο προθέρμανσης κόμβων μέσω DaemonSet. Περιλαμβάνει δύο περιέκτες (`pause-phi3` και `pause-mistral`) ανά pod, έναν για κάθε εικόνα περιέκτη.
- `locust-configmap.yaml` και `locust-deployment.yaml` — ανάπτυξη του Locust εντός του υπολογιστικού συμπλέγματος, ώστε οι μετρήσεις να μην επηρεάζονται από εξωτερική δικτυακή επικοινωνία.

5.8 Makefile

Το Makefile ορίζει τις παρακάτω εντολές:

- `make build` — κατασκευάζει τις εικόνες backend και gradio (έκδοση `$(VERSION)`).
- `make push` — push των παραπάνω στο Docker Hub.
- `make build-ollama` / `make push-ollama` — κατασκευάζει και δημοσιεύει την προδημιουργημένη εικόνα Ollama για το Phi-3 Mini.
- `make build-mistral` / `push-mistral` — αντίστοιχα για την εικόνα Mistral.
- `make deploy` — εφαρμόζει όλα τα αρχεία δηλωτικής διαμόρφωσης (manifests) στη σωστή σειρά (`config/secret` → `DaemonSet` → `storage/db/app` → `services` → `HPA` → `Locust`), χρησιμοποιώντας `sed` για αντικατάσταση των placeholders `__VERSION__`, `__OLLAMA_VERSION__` και `__MISTRAL_VERSION__`.
- `make status` / `rollback` εργαλεία επισκόπησης της κατάστασης και επαναφοράς σε προηγούμενη έκδοση.

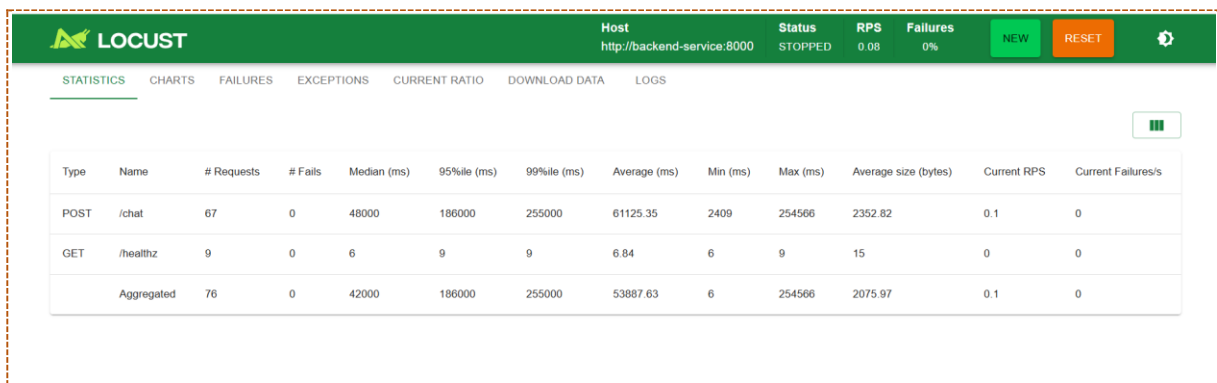
Το σχήμα εκδόσεων (version scheme) διαχωρίζει τις εκδόσεις του Backend και του Gradio (`VERSION`) από τις εκδόσεις των εικόνων Ollama (`OLLAMA_VERSION`, `MISTRAL_VERSION`), ώστε αλλαγές στον κώδικα της εφαρμογής να μην προκαλούν εκ νέου δημιουργία εικόνων μεγέθους περίπου 5 GB.

5.9 Ρύθμιση Locust

Το αρχείο `locust/locustfile.py` ορίζει τη συμπεριφορά των εικονικών χρηστών:

- Χρόνος αναμονής μεταξύ αιτημάτων: 1–5 δευτερόλεπτα, ώστε να προσομοιώνεται ο χρόνος σκέψης ενός πραγματικού χρήστη.
- Εργασίες: POST /chat (βάρος 10), GET /healthz (βάρος 3) και GET /metrics (βάρος 1).
- Κεφαλίδα Connection: close σε όλα τα αιτήματα. Χωρίς αυτή, η επαναχρησιμοποίηση μόνιμων συνδέσεων HTTP/1.1 από κάθε εργαζόμενο Locust θα μπορούσε να οδηγήσει σε προσκόλληση αιτημάτων στο ίδιο pod, λόγω της εξισορρόπησης φορτίου σε επίπεδο σύνδεσης από το kube-proxy.
- Σύνολο προκαθορισμένων ερωτημάτων (query bank): εννέα εταιρικά ερωτήματα που αφορούν το Project Atlas, το τμήμα Engineering, επαναφορά κωδικού VPN, πολιτική ετήσιας άδειας και προτεραιότητες του Μαρτίου 2026.

Εικόνα 10: Στατιστικά δοκιμών φόρτου στο Locust.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/chat	67	0	48000	186000	255000	61125.35	2409	254566	2352.82	0.1	0
GET	/healthz	9	0	6	9	9	6.84	6	9	15	0	0
Aggregated		76	0	42000	186000	255000	53887.63	6	254566	2075.97	0.1	0

ΚΕΦΑΛΑΙΟ 6 Πειραματική Αξιολόγηση

6.1 Πειραματική Διάταξη

Όλα τα πειράματα εκτελέστηκαν στο υπολογιστικό σύμπλεγμα vdccloud του Τμήματος, σε υποδομή αποκλειστικής χρήσης CPU. Όπου παρακάτω αναφέρονται μετρήσεις χρόνου ή καθυστέρησης, αφορούν αυτή τη διάταξη.

- Υπολογιστικό σύμπλεγμα: vdccloud, χώρος ονομάτων vtheofil-priv.
- Backend pod: ένας περιέκτης FastAPI και ένας παράπλευρος περιέκτης Ollama.
- HPA: minReplicas: 1, maxReplicas: 5, στόχος μέσης αξιοποίησης CPU 70%.
- ChromaDB: ένα αντίγραφο με μόνιμο αποθηκευτικό χώρο μέσω PVC μεγέθους 2 GiB.
- Locust: ανάπτυξη εντός του υπολογιστικού συμπλέγματος, ώστε οι μετρήσεις να μην επηρεάζονται από εξωτερική δικτυακή επικοινωνία.

Εικόνα 11: Έξοδος της εντολής `kubectl get pods -n vtheofil-priv` πριν από τη δοκιμή φόρτου.

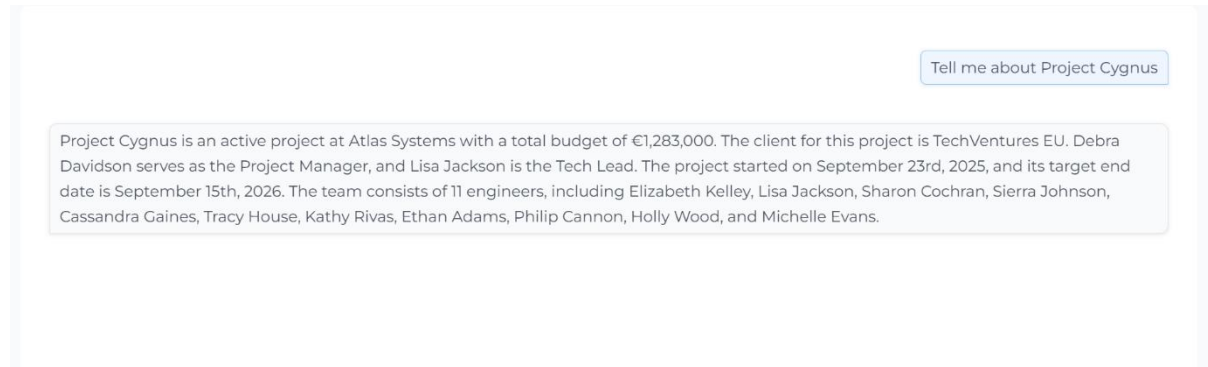
```
vtheofil@DESKTOP-J11N11M:~$ kubectl get pods
NAME                                READY   STATUS
backend-deployment-6f6d6d7f7b-p6xtk 2/2     Running
chromadb-deployment-6dbcb59cc8-jbm2r 1/1     Running
gradio-deployment-5fcd9fd7d8-l9t54 1/1     Running
locust-deployment-684ddfb5bd-r9bsr 1/1     Running
ollama-image-preloader-4nkzt 2/2     Running
ollama-image-preloader-5h6hp 2/2     Running
ollama-image-preloader-5j6t5 2/2     Running
ollama-image-preloader-c8dzd 2/2     Running
ollama-image-preloader-cp5f2 2/2     Running
ollama-image-preloader-f2x5g 2/2     Running
ollama-image-preloader-ghlt6 2/2     Running
ollama-image-preloader-gmvt5 2/2     Running
ollama-image-preloader-h7cjt 2/2     Running
ollama-image-preloader-j4j6v 2/2     Running
ollama-image-preloader-jxv8t 2/2     Running
ollama-image-preloader-nfbfp 2/2     Running
ollama-image-preloader-pxnc9 2/2     Running
ollama-image-preloader-qh4qw 2/2     Running
ollama-image-preloader-sd4cr 2/2     Running
ollama-image-preloader-sxsrl 2/2     Running
ollama-image-preloader-vmpp9 2/2     Running
ollama-image-preloader-w7j29 2/2     Running
ollama-image-preloader-w7jfk 2/2     Running
ollama-image-preloader-wkk9f 2/2     Running
```

6.2 Λειτουργική Επιβεβαίωση του RAG

Πριν από τα πειράματα φόρτου, επιβεβαιώθηκε η ορθή λειτουργία της διαδικασίας RAG μέσω χειροκίνητων ερωτημάτων στη διεπαφή χρήστη Gradio. Παραδείγματα επιτυχημένης ανάκτησης σχετίζονται με ερωτήσεις όπως: «Tell me about Project Cygnus» — η απάντηση περιείχε σωστά τον προϋπολογισμό, τον διαχειριστή έργου και τον τεχνικό υπεύθυνο, τις ημερομηνίες έναρξης και προγραμματισμένου τέλους, καθώς και τα βασικά στοιχεία του έργου.

Σημαντική παρατήρηση ήταν ότι ο μηχανισμός φιλτραρίσματος βάσει πρόθεσης και μεταδεδομένων (`where={"type": "project"}`) ήταν απαραίτητος, ώστε να αποκλείονται τμήματα από αρχεία μηνιαίων πλάνων που απλώς ανέφεραν το όνομα του έργου. Χωρίς αυτό το φίλτρο, το γλωσσικό μοντέλο ανακαλούσε αναμεμιγμένο περιεχόμενο από τα δύο είδη εγγράφων και η απάντηση γινόταν διφορούμενη.

Εικόνα 12: Λειτουργική επιβεβαίωση του συστήματος RAG.



Εικόνα 13: Παράδειγμα ανακτημένου τμήματος κειμένου.



6.3 Δοκιμές Φόρτου: Phi-3 Mini vs Mistral 7B

Η κύρια συγκριτική αξιολόγηση των δύο μοντέλων πραγματοποιήθηκε με το Locust, με τρεις ισοδύναμες δοκιμές φόρτου για κάθε μοντέλο: 3, 5 και 10 ταυτόχρονους χρήστες, αντίστοιχα, με διάρκεια 10–15 λεπτών ανά δοκιμή. Όλες οι μετρήσεις προέρχονται από το υπολογιστικό σύμπλεγμα vnccloud, σε υποδομή αποκλειστικής χρήσης CPU, με διαμόρφωση HPA minReplicas: 1, maxReplicas: 5 και στόχο μέσης αξιοποίησης CPU 70%.

6.3.1 Μεθοδολογία Καθαρού Σημείου Εκκίνησης (Baseline)

Πριν από κάθε δοκιμή, το Deployment της υπηρεσίας υποστήριξης επανερχόταν σε κατάσταση ενός αντιγράφου. Στη συνέχεια, το υφιστάμενο pod διαγραφόταν, ώστε να δημιουργηθεί νέο pod χωρίς υπολειπόμενη κατάσταση μνήμης από προηγούμενες δοκιμές.

Ακολουθούσε αναμονή περίπου 90 δευτερολέπτων για την εκκίνηση και την προθέρμανση του νέου pod Phi-3 Mini ή Mistral 7B, καθώς και για τη φόρτωση του μοντέλου στην κύρια μνήμη. Μετά την ολοκλήρωση της διαδικασίας αυτής ξεκινούσε η δοκιμή φόρτου με το Locust.

Η μεθοδολογία αυτή περιόριζε την επίδραση υπολειπόμενης κατάστασης από προηγούμενες δοκιμές και εξασφάλιζε ότι κάθε δοκιμή ξεκινούσε υπό συγκρίσιμες πειραματικές συνθήκες.

6.3.2 Αποτελέσματα: 3 ταυτόχρονοι χρήστες

Με 3 ταυτόχρονους χρήστες (M1 για Mistral, διάρκεια 10 λεπτά · P1-clean για Phi-3, διάρκεια 10 λεπτά), τα αποτελέσματα συνοψίζονται στον Πίνακα 6.1:

Μετρική	Mistral 7B (M1)	Phi-3 Mini (P1)
Αιτήματα	9	23
Αποτυχίες	0%	0%
Διάμεση καθυστέρηση	187 s	56 s
Καθυστέρηση p95	324 s	121 s
Καθυστέρηση p99	324 s	134 s
Μέση καθυστέρηση	181 s	69 s
Ρυθμός εξυπηρέτησης (RPS)	0.02	0.10
Τυπική κατανάλωση μνήμης	~5.0 GiB	~3.8 GiB

Πίνακας 1 Σύγκριση Phi-3 Mini και Mistral 7B στα 3 ταυτόχρονους χρήστες (CPU-only).

Στη δοκιμή με 3 ταυτόχρονους χρήστες, το Phi-3 Mini εξυπηρέτησε 2,56 φορές περισσότερα αιτήματα στο ίδιο χρονικό παράθυρο, με 3,34 φορές χαμηλότερη διάμεση καθυστέρηση και περίπου πέντε φορές υψηλότερο ρυθμό εξυπηρέτησης. Η κατανάλωση μνήμης παρέμεινε περίπου 25% χαμηλότερη. Και τα δύο μοντέλα διατήρησαν ποσοστό αποτυχιών 0%.

Εικόνα 14: Στατιστικά Locust για Phi-3 Mini, δοκιμή P1 με 3 ταυτόχρονους χρήστες και διάρκεια 10 λεπτών.

LOCUST

Host

http://backend-service:8000

Status

STOPPED

RPS

0.04

Failures

0%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

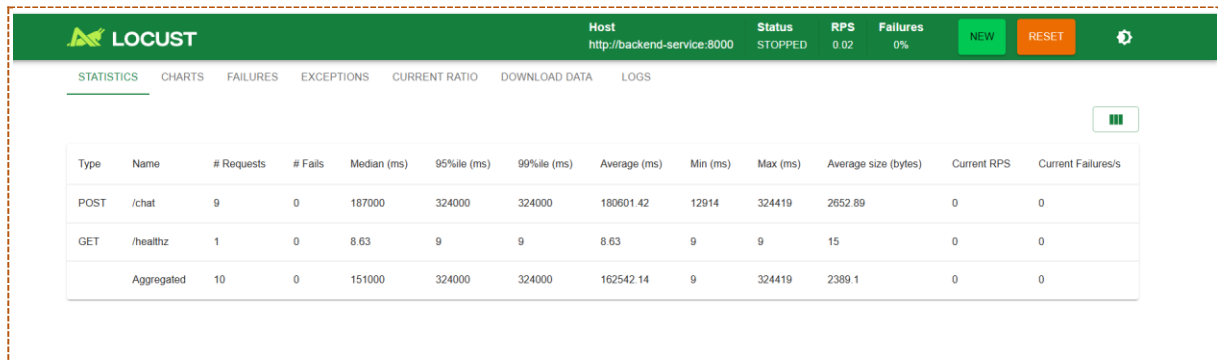
CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/chat	23	0	56000	121000	134000	69277.25	31523	134218	2284.43	0.1	0
GET	/healthz	1	0	6.67	7	7	6.67	7	7	15	0	0
	Aggregated	24	0	54000	121000	134000	66390.97	7	134218	2189.88	0.1	0

Εικόνα 15: Στατιστικά Locust για Mistral 7B, δοκιμή M1 με 3 ταυτόχρονους χρήστες και διάρκεια 10 λεπτών.



6.3.3 Αποτελέσματα: 5 ταυτόχρονοι χρήστες

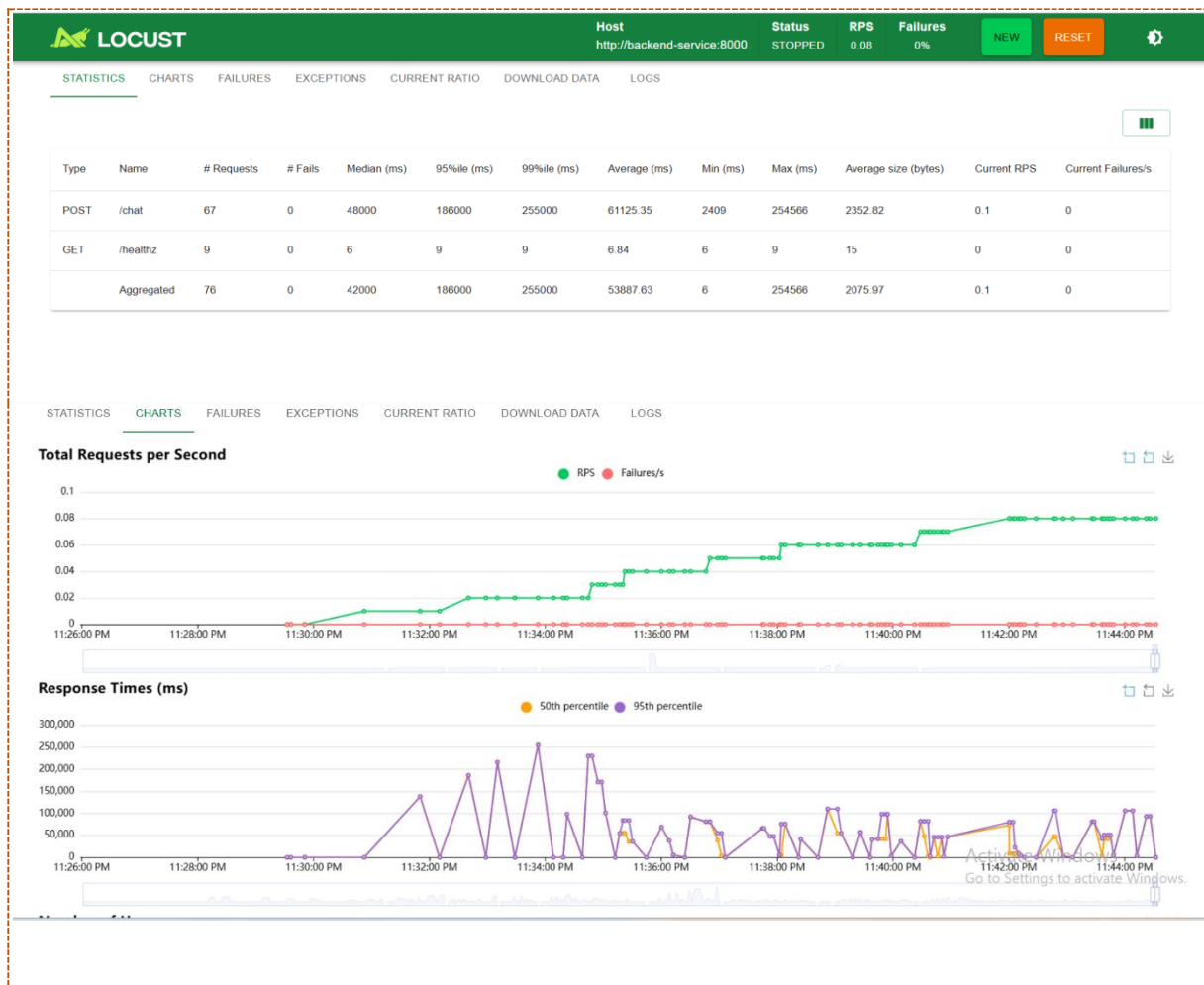
Με 5 ταυτόχρονους χρήστες (M2 και P2, αμφότερα 15 λεπτά), το χάσμα διευρύνεται (Πίνακας 6.2):

Μετρική	Mistral 7B (M2)	Phi-3 Mini (P2)
Αιτήματα	20	67
Αποτυχίες	0%	0%
Διάμεση καθυστέρηση	177 s	48 s
Καθυστέρηση p95	396 s	186 s
Μέση καθυστέρηση	187 s	61 s
Ρυθμός εξυπηρέτησης (RPS)	0.022	0.074
Τυπική κατανάλωση μνήμης	7.9 GiB	4.6 GiB

Πίνακας 2 Σύγκριση Phi-3 Mini και Mistral 7B στα 5 ταυτόχρονους χρήστες, διάρκεια 15 λεπτών.

Το Phi-3 Mini εξυπηρέτησε 3,35 φορές περισσότερα αιτήματα σε 15 λεπτά (67 έναντι 20) και η διάμεση καθυστέρηση ήταν 3,69 φορές χαμηλότερη. Παρόλο που και τα δύο μοντέλα έφτασαν στο όριο των 5 αντιγράφων του HPA, το Phi-3 Mini αξιοποίησε καλύτερα την υπάρχουσα χωρητικότητα, όπως αποτυπώνεται στις σημαντικά χαμηλότερες καθυστερήσεις και στον υψηλότερο ρυθμό εξυπηρέτησης. Η μέγιστη κατανάλωση μνήμης του Phi-3 Mini παρέμεινε στα 4,6 GiB, έναντι 7,9 GiB του Mistral 7B, δηλαδή περίπου 42% χαμηλότερη.

Εικόνα 16: Στατιστικά και διαγράμματα Locust για Phi-3 Mini, δοκιμή P2 με 5 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.



*Εικόνα 17: Χρήση πόρων των pods κατά τη δοκιμή Phi-3 P2 μέσω της εντολής
kubectl top pods.*

Every 5.0s:DESKTOP-J11N11M: Tue Jun 16 23:31:49 2

=== 23:31:49 ===

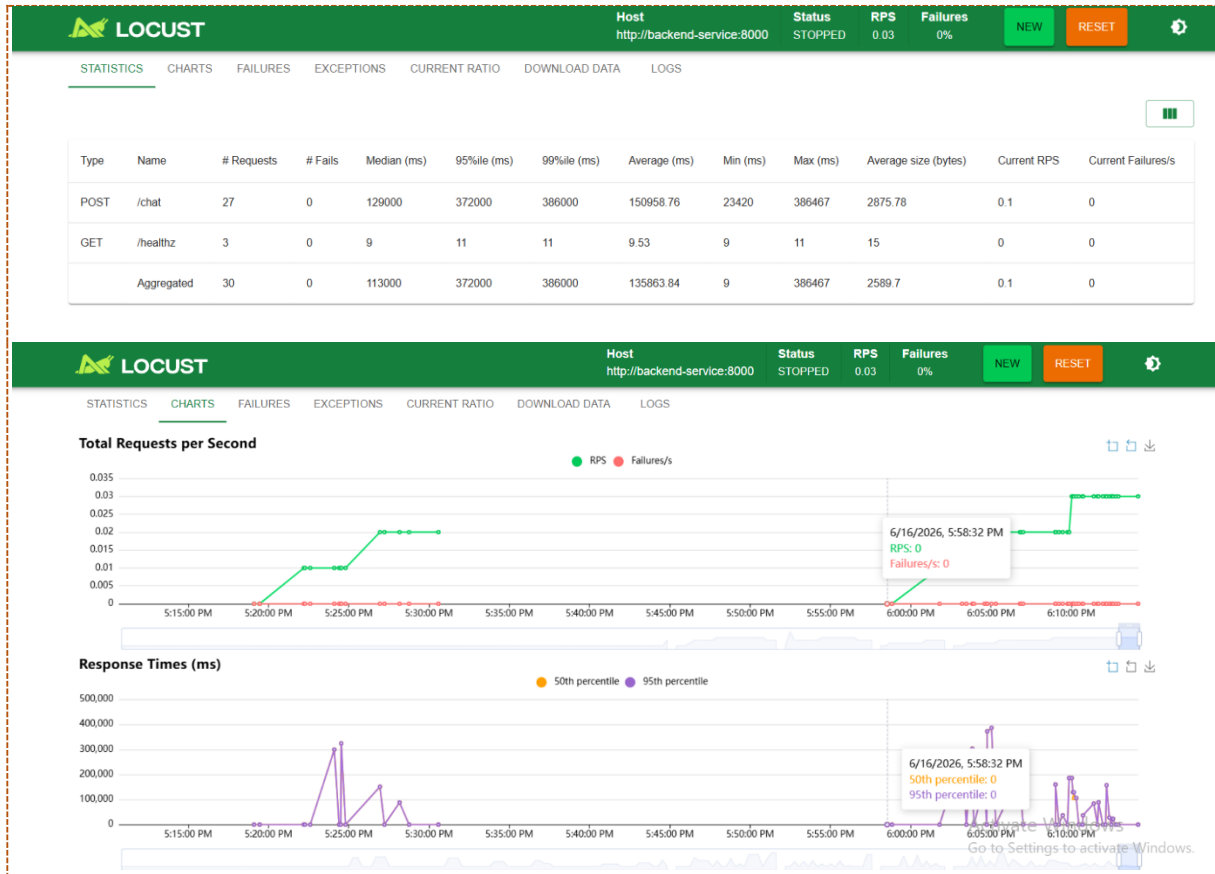
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
backend-hpa	Deployment/backend-deployment	cpu: 159%/70%	1	5	3	6d23h
NAME	READY	STATUS	RESTARTS	AGE		
backend-deployment-6f6d6d7f7b-8r5mq	1/2	Running	0	96s		
backend-deployment-6f6d6d7f7b-d7rw7	1/2	Running	0	95s		
backend-deployment-6f6d6d7f7b-jvtf4	2/2	Running	0	6m39s		
backend-deployment-6f6d6d7f7b-8r5mq	ollama	1m	3688Mi			
backend-deployment-6f6d6d7f7b-d7rw7	ollama	1m	3685Mi			
backend-deployment-6f6d6d7f7b-jvtf4	ollama	3985m	3769Mi			
ollama-image-preloader-4nkzt	pause-mistral	0m	0Mi			

Every 5.0s:DESKTOP-J11N11M: Tue Jun 16 23:36:03

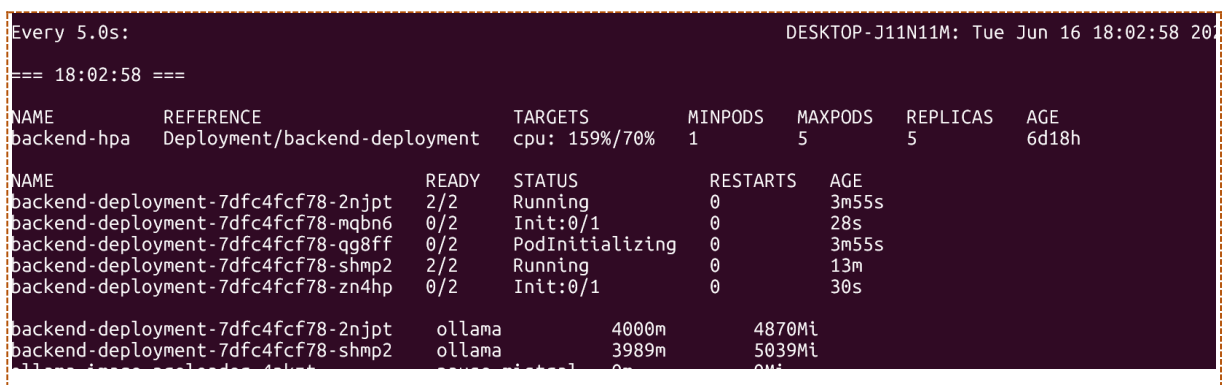
=== 23:36:03 ===

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
backend-hpa	Deployment/backend-deployment	cpu: 96%/70%	1	5	5	6d23h
NAME	READY	STATUS	RESTARTS	AGE		
backend-deployment-6f6d6d7f7b-8r5mq	2/2	Running	0	5m50s		
backend-deployment-6f6d6d7f7b-d7rw7	2/2	Running	0	5m49s		
backend-deployment-6f6d6d7f7b-jvtf4	2/2	Running	0	10m		
backend-deployment-6f6d6d7f7b-s9h9s	2/2	Running	0	3m3s		
backend-deployment-6f6d6d7f7b-t7sbp	2/2	Running	0	3m3s		
backend-deployment-6f6d6d7f7b-8r5mq	ollama	3993m	4152Mi			
backend-deployment-6f6d6d7f7b-d7rw7	ollama	1m	4203Mi			
backend-deployment-6f6d6d7f7b-jvtf4	ollama	2m	4577Mi			
backend-deployment-6f6d6d7f7b-s9h9s	ollama	3996m	3728Mi			
backend-deployment-6f6d6d7f7b-t7sbp	ollama	3997m	3744Mi			
ollama-image-preloader-4nkzt	pause-mistral	0m	0Mi			

Εικόνα 18: Στατιστικά Locust για Mistral 7B με 5 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.



Εικόνα 19: Χρήση πόρων των pods κατά τη δοκιμή Mistral 7B με 5 ταυτόχρονους χρήστες μέσω της εντολής `kubectl top pods`.



```
every 5.0s:
DESKTOP-J11N11M: Tue Jun 16 18:10:34 2026
=== 18:10:34 ===

NAME      REFERENCE      TARGETS      MINPODS      MAXPODS      REPLICAS      AGE
backend-hpa  Deployment/backend-deployment  cpu: 95%/70%  1            5            5            6d18h

NAME      READY      STATUS      RESTARTS      AGE
backend-deployment-7dfc4fcf78-2njpt  2/2      Running    0            11m
backend-deployment-7dfc4fcf78-mqbn6  2/2      Running    0            8m2s
backend-deployment-7dfc4fcf78-qg8ff  2/2      Running    0            11m
backend-deployment-7dfc4fcf78-shmp2  2/2      Running    0            21m
backend-deployment-7dfc4fcf78-zn4hp  2/2      Running    0            8m4s

backend-deployment-7dfc4fcf78-2njpt  ollama      3999m      5276Mi
backend-deployment-7dfc4fcf78-mqbn6  ollama      3924m      5010Mi
backend-deployment-7dfc4fcf78-qg8ff  ollama      3395m      5884Mi
backend-deployment-7dfc4fcf78-shmp2  ollama      3990m      5162Mi
backend-deployment-7dfc4fcf78-zn4hp  ollama      3801m      4882Mi
ollama-image-preloader-4nkzt  pause-mistral  0m        0Mi
```

6.3.4 Αποτελέσματα: 10 ταυτόχρονοι χρήστες

Η δοκιμή με 10 ταυτόχρονους χρήστες ανέδειξε την κρισιμότερη διαφορά μεταξύ των δύο μοντέλων. Με maxReplicas: 5 και 10 ταυτόχρονους χρήστες, το σύστημα λειτουργεί υπό σαφή πίεση ουράς αιτημάτων (queue back-pressure). Τα αποτελέσματα του Πίνακα 6.3 αναδεικνύουν το σημείο καμψής (breaking point) του Mistral:

Μετρική	Mistral 7B (M4)	Phi-3 Mini (P3)
Αιτήματα	19	89
Αποτυχίες	0%	0%
Διάμεση καθυστέρηση	192 s	76 s
Καθυστέρηση p95	831 s (13.85 min)	263 s (4.4 min)
Καθυστέρηση p99	831 s	355 s
Μέση καθυστέρηση	349 s	93 s
Μέγιστη καθυστέρηση	831 s	355 s
Ρυθμός εξυπηρέτησης (RPS)	0.02	0.099

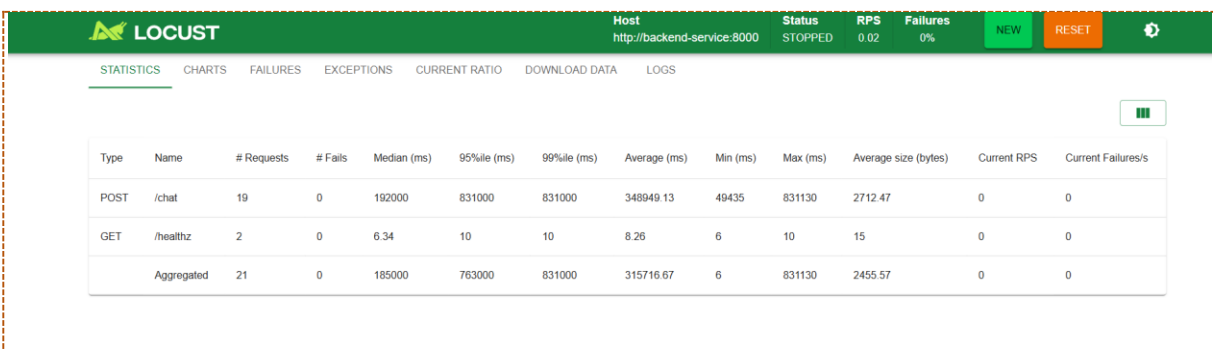
Πίνακας 3 Σύγκριση Phi-3 Mini και Mistral 7B στα 10 ταυτόχρονους χρήστες — το breaking point του Mistral.

Στη δοκιμή με 10 χρήστες, το Mistral παρουσίασε κατάρρευση του ρυθμού εξυπηρέτησης: ο RPS παρέμεινε στο 0,02, δηλαδή στην ίδια τιμή με τη δοκιμή των 3 χρηστών. Αυτό αποτελεί ένδειξη ότι το σύστημα είχε φτάσει στο όριο των 5 αντιγράφων του HPA, ενώ τα διαθέσιμα pods δεν μπορούσαν να εξυπηρετήσουν περισσότερα αιτήματα με αποδεκτή καθυστέρηση. Η καθυστέρηση p95 αυξήθηκε στα 831 δευτερόλεπτα, δηλαδή περίπου 13,85 λεπτά. Παρόλο που δεν εμφανίστηκαν αποτυχίες, η εμπειρία χρήστη ήταν πρακτικά μη αποδεκτή.

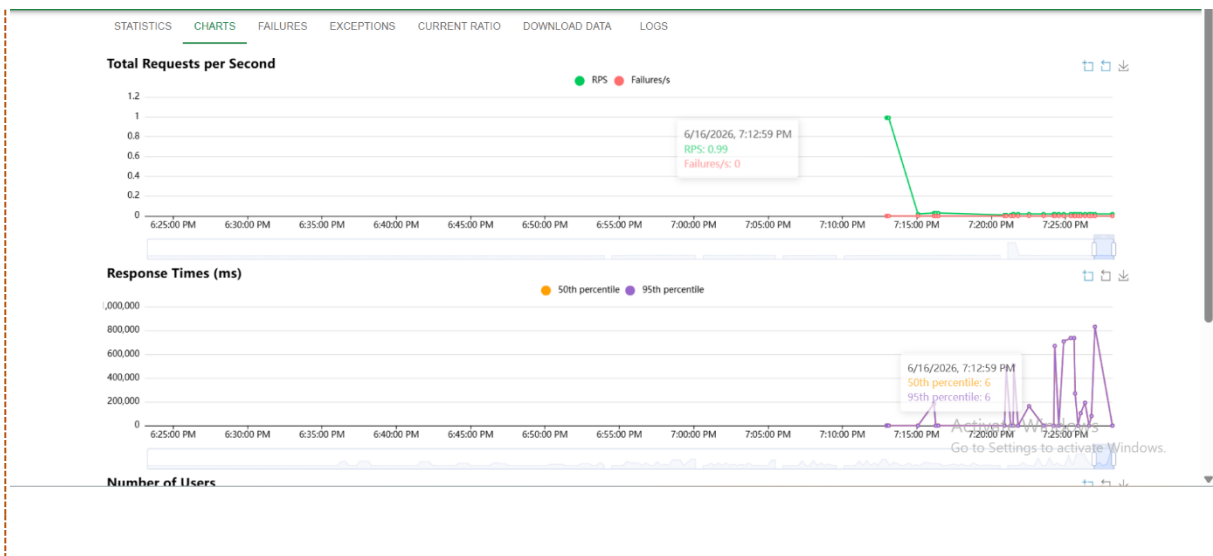
Αντίθετα, το Phi-3 Mini διατήρησε λειτουργικά αποδεκτότερες επιδόσεις: ολοκλήρωσε 4,68 φορές περισσότερα αιτήματα (89 έναντι 19), με διάμεση καθυστέρηση 76 δευτερολέπτων και καθυστέρηση p95 263 δευτερολέπτων. Με χαμηλότερη μέγιστη κατανάλωση μνήμης (4,8 GiB έναντι περίπου 7 GiB για το Mistral), το Phi-3 Mini δείχνει ότι η λειτουργική χωρητικότητα της υποδομής επηρεάζεται ουσιαστικά από την επιλογή του μοντέλου, χωρίς να απαιτείται αλλαγή στη διαμόρφωση του HPA.

.

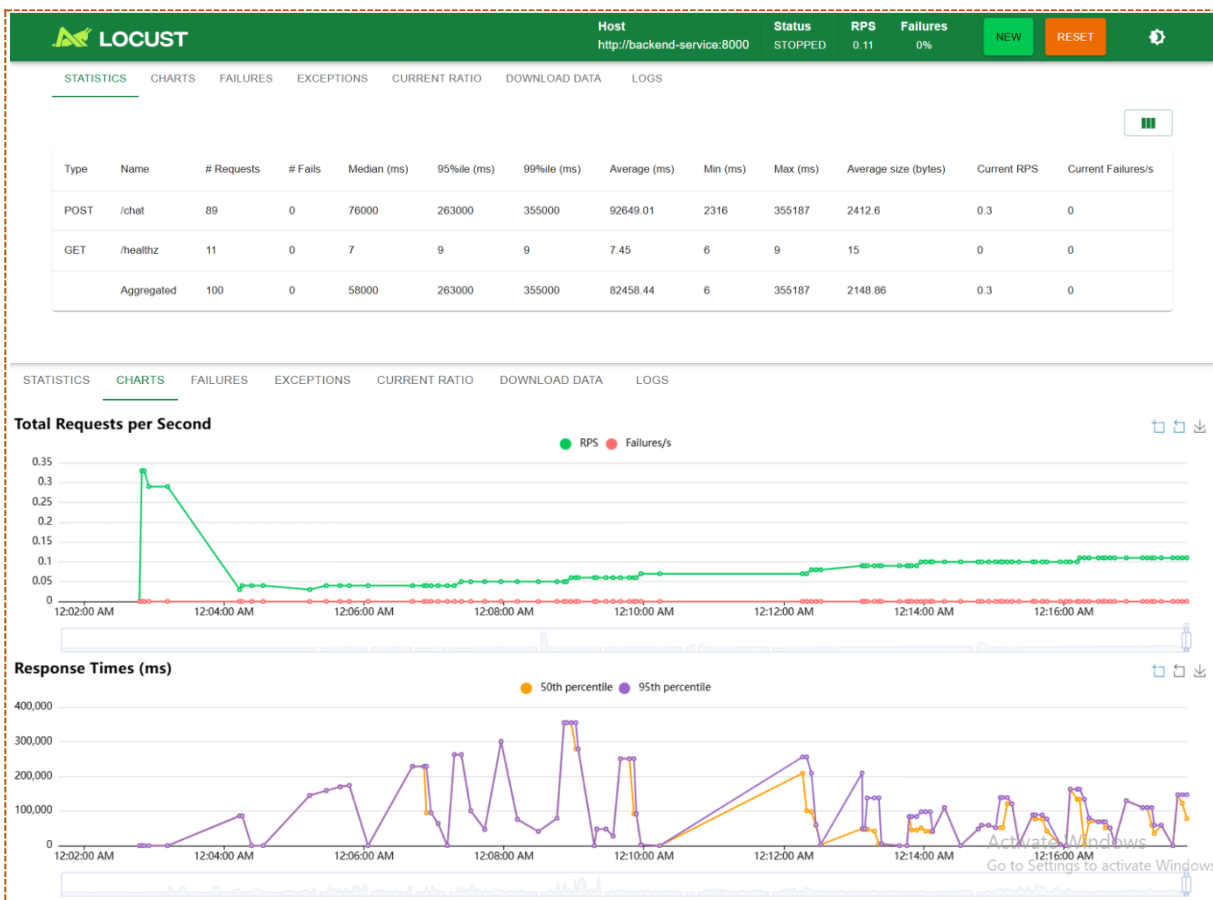
Εικόνα 20: Στατιστικά και διαγράμματα Locust για Mistral 7B, δοκιμή M3 με 10 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/chat	19	0	192000	831000	831000	348949.13	49435	831130	2712.47	0	0
GET	/healthz	2	0	6.34	10	10	8.26	6	10	15	0	0
	Aggregated	21	0	185000	763000	831000	315716.67	6	831130	2455.57	0	0



Εικόνα 21: Στατιστικά και διαγράμματα Locust για Phi-3 Mini, δοκιμή P3 με 10 ταυτόχρονους χρήστες και διάρκεια 15 λεπτών.



6.3.5 Παρατηρήσεις για τον Κορεσμό του Ρυθμού Εξυπηρέτησης και την Άνιση Κατανομή Φορτίου

Παρατηρήθηκαν δύο φαινόμενα στις δοκιμές και των δύο μοντέλων:

- Κορεσμός του ρυθμού εξυπηρέτησης με 5 ή περισσότερους χρήστες: παρόλο που οι χρήστες αυξάνονταν από 5 σε 10, η συνολική ρυθμοαπόδοση αυξανόταν οριακά (Phi-3 Mini: 0,074 → 0,099 RPS, Mistral 7B: 0,022 → 0,020 RPS). Όταν ο HPA έφτανε το maxReplicas: 5, η περαιτέρω αύξηση του φόρτου δεν οδηγούσε σε αντίστοιχη αύξηση του ρυθμού εξυπηρέτησης. Η απόδοση περιοριζόταν από τη διαθέσιμη χωρητικότητα της εφαρμογής υπό τη συγκεκριμένη διαμόρφωση HPA και τους διαθέσιμους πόρους CPU.
- Μη ομοιόμορφη κατανομή των αιτημάτων μεταξύ των αντιγράφων: σε στιγμιότυπα των δοκιμών με 5 και 10 χρήστες, ορισμένα διαθέσιμα αντίγραφα παρουσίαζαν πολύ χαμηλή χρήση CPU, ενώ άλλα προσέγγιζαν τα 4.000 mCPU. Το εύρημα αυτό δείχνει ότι η διαθέσιμη χωρητικότητα δεν αξιοποιούνταν πάντοτε ισομερώς. Η χρήση της κεφαλίδας Connection: close περιόρισε την προσκόλληση αιτημάτων σε μόνιμες συνδέσεις, χωρίς όμως να εγγυάται πλήρως ομοιόμορφη κατανομή σε όλα τα pods.

Τα παραπάνω ευρήματα δικαιολογούν τη διερεύνηση πιο εξειδικευμένων μηχανισμών δρομολόγησης επιπέδου εφαρμογής (L7), καθώς και κλιμάκωσης βάσει ρυθμού αιτημάτων ή μήκους ουράς μέσω εργαλείων όπως το KEDA, ως μελλοντική εργασία

.

6.4 Συμπεριφορά του HPA

Κατά τη διάρκεια των δοκιμών φόρτου, ο HPA παρακολουθήθηκε μέσω των εντολών `kubectl get hpa` και `kubectl describe hpa`. Η συμπεριφορά του, ως προς τις αποφάσεις κλιμάκωσης, ήταν ορθή: όταν η μέση αξιοποίηση CPU ξεπερνούσε την τιμή-στόχο του 70%, ο HPA αύξανε τον

επιθυμητό αριθμό αντιγράφων. Ο αριθμός των αντιγράφων αυξανόταν σταδιακά από 1 σε 2, 3, 4 και 5 μέσα σε λίγα λεπτά.

Ωστόσο, η δημιουργία νέων αντιγράφων δεν σήμαινε ότι αυτά ήταν άμεσα διαθέσιμα για εξυπηρέτηση αιτημάτων, καθώς απαιτούνταν η εκκίνηση του pod και η προθέρμανση του μοντέλου. Επομένως, η μηχανική λειτουργία του HPA ήταν ορθή, αλλά η συνολική αποτελεσματικότητα της κλιμάκωσης επηρεαζόταν από τον χρόνο ψυχρής εκκίνησης και την κατανομή του φόρτου.

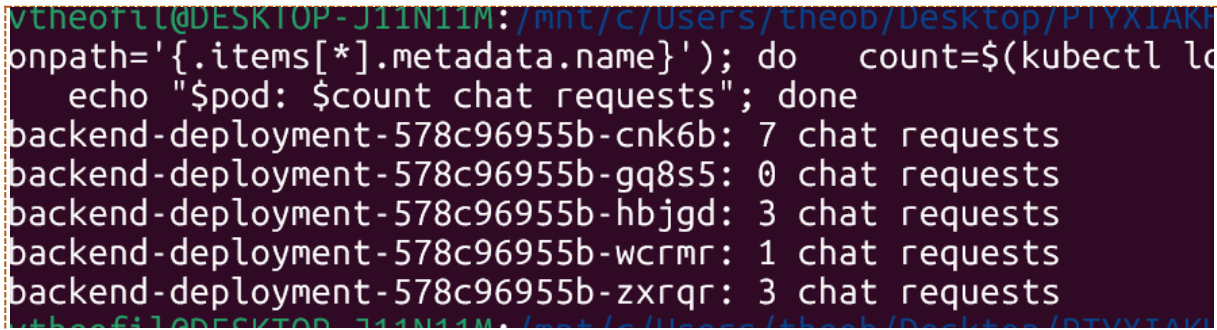
6.4.1 Παρατηρήσεις σχετικά με την κατανομή φορτίου

Κατά τη διάρκεια των δοκιμών παρατηρήθηκε ότι η κατανομή των αιτημάτων μεταξύ των αντιγράφων δεν ήταν απολύτως ομοιόμορφη. Στα αρχικά στάδια μιας δοκιμής φόρτου, το πρώτο διαθέσιμο αντίγραφο συχνά δεχόταν μεγαλύτερο ποσοστό των αιτημάτων, ενώ τα νεότερα αντίγραφα βρίσκονταν ακόμη στη διαδικασία εκκίνησης ή προθέρμανσης του μοντέλου.

Αφού τα νέα pods ολοκλήρωναν τον έλεγχο ετοιμότητας, η κίνηση άρχιζε να κατανέμεται και σε πολλαπλά διαθέσιμα αντίγραφα, χωρίς όμως να επιτυγχάνεται απόλυτα ισομερής κατανομή. Ορισμένα αντίγραφα εξυπηρετούσαν περισσότερα αιτήματα από άλλα, γεγονός που μπορεί να σχετίζεται με την κατανομή σε επίπεδο σύνδεσης και με τις διαφορετικές χρονικές στιγμές κατά τις οποίες κάθε pod γινόταν διαθέσιμο.

Επομένως, η αύξηση των αντιγράφων μέσω του HPA οδηγούσε σε πραγματική αξιοποίηση επιπλέον υπολογιστικών πόρων. Παρ' όλα αυτά, η αποτελεσματική χωρητικότητα της υποδομής παρέμενε μικρότερη από τη θεωρητική, λόγω του χρόνου προθέρμανσης των LLM και της μη πλήρως ισομερούς κατανομής των αιτημάτων μεταξύ των διαθέσιμων αντιγράφων.

Εικόνα 22: Κατανομή αιτημάτων συνομιλίας μεταξύ πέντε ενεργών pods.



```
theofil@DESKTOP-J11N11M: /mnt/c/Users/theob/Desktop/ΠΤΥΧΙΑΚΗ
onpath='{.items[*].metadata.name}'); do count=$(kubectl get pods --selector=chat-backend --output=table | grep $pod | wc -l); echo "$pod: $count chat requests"; done
backend-deployment-578c96955b-cn66b: 7 chat requests
backend-deployment-578c96955b-gq8s5: 0 chat requests
backend-deployment-578c96955b-hbjgd: 3 chat requests
backend-deployment-578c96955b-wcrmr: 1 chat requests
backend-deployment-578c96955b-zxrqr: 3 chat requests
```

6.4.2 Ανατομία Ψυχρής Εκκίνησης (Cold-Start Anatomy)

Σε μια καταγεγραμμένη παρατήρηση, ένα νέο pod της υπηρεσίας υποστήριξης δημιουργήθηκε στη χρονοσήμανση 01:36:10Z και έφτασε σε κατάσταση Ready (2/2) στη χρονοσήμανση 01:43:32Z, δηλαδή 7 λεπτά και 22 δευτερόλεπτα αργότερα. Η ανάλυση του χρόνου αυτού ήταν η ακόλουθη:

- Χρήση της ήδη διαθέσιμης εικόνας περιέκτη και εκκίνηση του περιέκτη Ollama: περίπου 30 δευτερόλεπτα.
- Εκκίνηση του περιέκτη και ετοιμότητα της υπηρεσίας Ollama: περίπου 10 δευτερόλεπτα.
- Λήψη του μοντέλου από το registry.ollama.ai: περίπου 6 λεπτά και 30 δευτερόλεπτα για το Mistral 7B μεγέθους 4,4 GB μέσω του δικτύου του υπολογιστικού συμπλέγματος.
- Ολοκλήρωση του προσαρμοσμένου ανιχνευτή ετοιμότητας: περίπου 12 δευτερόλεπτα.

Η μέτρηση αυτή αφορά διάταξη χωρίς προδημιουργημένη εικόνα και χωρίς προθέρμανση κόμβων μέσω DaemonSet. Με την ενεργοποίηση των δύο αυτών μοτίβων, η λήψη του μοντέλου εξαλείφεται, ενώ η εικόνα περιέκτη είναι ήδη διαθέσιμη στους κόμβους στους οποίους έχει εκτελεστεί το DaemonSet. Ο χρόνος μέχρι την κατάσταση ετοιμότητας μειώνεται σε περίπου 30–60 δευτερόλεπτα και αφορά κυρίως την

εκκίνηση των περιεκτών, τον έλεγχο ετοιμότητας και την προθέρμανση του μοντέλου μέσω του μηχανισμού postStart.

6.4.3 Βελτιστοποίηση Ψυχρής Εκκίνησης: Διόρθωση Επανεισαγωγής Δεδομένων από τον Περιέκτη Αρχικοποίησης

Κατά τη διάρκεια των πειραμάτων εντοπίστηκε μια λανθάνουσα πηγή καθυστέρησης στην ψυχρή εκκίνηση. Ο περιέκτης αρχικοποίησης εκτελούσε την εντολή `python -m app.ingest --force` σε κάθε εκκίνηση pod, διαγράφοντας και επανεισάγοντας τα 499 τμήματα κειμένου της συλλογής ChromaDB κάθε φορά που ο HPA δημιουργούσε νέο αντίγραφο. Η συμπεριφορά αυτή είχε δύο επιπτώσεις:

- Προσθήκη περίπου 2 λεπτών στην ψυχρή εκκίνηση κάθε νέου αντιγράφου, καθώς η δημιουργία διανυσματικών αναπαραστάσεων για 499 τμήματα απαιτούσε περίπου 60–90 δευτερόλεπτα στην υποδομή CPU.
- Συνθήκη ανταγωνισμού (race condition): όσο το νέο pod επανεισήγαγε τα δεδομένα στη συλλογή, τα ήδη ενεργά pods που εξυπηρετούσαν αιτήματα μπορούσαν να βλέπουν προσωρινά κενή συλλογή, με αποτέλεσμα εσφαλμένες ή κενές απαντήσεις.

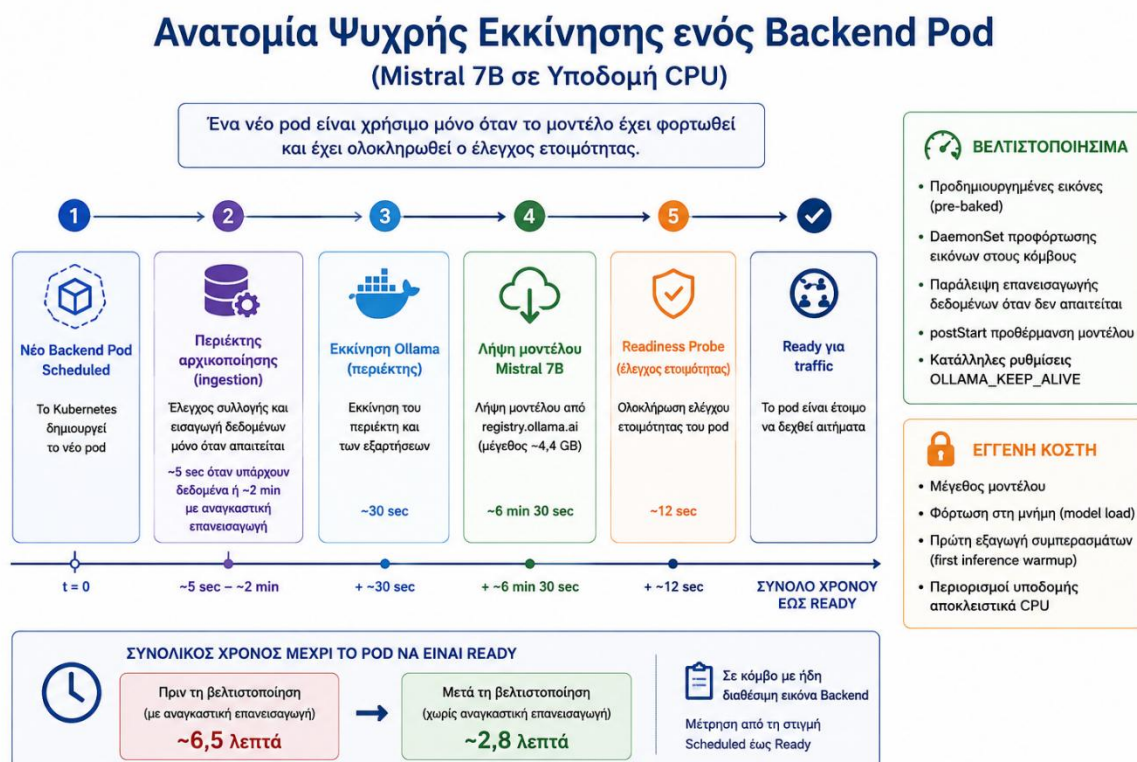
Η αρχική σχεδίαση χρησιμοποιούσε τη σημαία `--force`, ώστε οι αλλαγές στο σύνολο δεδομένων να εφαρμόζονται αυτόματα κατά την ανάπτυξη νέας έκδοσης. Ωστόσο, κατά την αυτόματη κλιμάκωση, όπου το σύνολο δεδομένων δεν αλλάζει, η σημαία αυτή αποτελούσε παράγοντα υποβάθμισης της απόδοσης. Η διόρθωση ήταν η αφαίρεση του `--force` από τον περιέκτη αρχικοποίησης. Το `ingest.py` υποστηρίζει ήδη έλεγχο `collection_has_data`, ο οποίος παραλείπει την εισαγωγή δεδομένων όταν η συλλογή περιέχει ήδη στοιχεία. Η αναγκαστική εισαγωγή δεδομένων πραγματοποιείται πλέον χειροκίνητα μέσω `kubectl exec`, μόνο όταν αλλάζει το σύνολο δεδομένων.

Πειραματική επίδραση της διόρθωσης (v2.0.9):

- Ψυχρή εκκίνηση Mistral, με την εικόνα του Backend ήδη διαθέσιμη στον κόμβο: από περίπου 6,5 λεπτά σε περίπου 2,8 λεπτά, με μέτρηση από Scheduled έως Ready.
- Χρόνος περιέκτη αρχικοποίησης: από περίπου 2 λεπτά για επανεισαγωγή 499 τμημάτων σε περίπου 5 δευτερόλεπτα, όταν η εισαγωγή παραλείπεται επειδή υπάρχουν ήδη δεδομένα.
- Εξάλειψη της συνθήκης ανταγωνισμού: τα ενεργά pods δεν βλέπουν πλέον κενή συλλογή κατά την κλιμάκωση προς τα πάνω.

Το συγκεκριμένο σημείο αποτελεί ένα από τα βασικά διδάγματα της εργασίας: οι περιέκτες αρχικοποίησης πρέπει να σχεδιάζονται με γνώμονα ότι ενδέχεται να εκτελεστούν πολλές φορές υπό συνθήκες αυτόματης κλιμάκωσης. Ενέργειες που θεωρούν ότι εκτελούνται μόνο μία φορά κατά την αρχική ανάπτυξη μπορούν να δημιουργήσουν προβλήματα όταν επαναλαμβάνονται σε γεγονότα κλιμάκωσης.

Εικόνα 23: Χρονική ακολουθία ψυχρής εκκίνησης νέου pod.



6.5 Συμπεριφορά του OLLAMA_KEEP_ALIVE υπό Φόρτο

Η ρύθμιση `OLLAMA_KEEP_ALIVE=24h` της Ενότητας 4.9 αξιολογήθηκε εμπειρικά σε δύο διαφορετικά σενάρια φόρτου, αναδεικνύοντας την αντιστάθμιση που συνεπάγεται η επιλογή της.

Σενάριο Α — αιχμιαίος φόρτος (επιτυχία της ρύθμισης): Σε μια καταγεγραμμένη παρατήρηση μετά από αιχμιαίο φόρτο 3 χρηστών, εντοπίστηκε ένα pod σε κατάσταση αδράνειας: η χρήση CPU είχε πέσει στα 3 mCPU, αλλά η δεσμευμένη κύρια μνήμη παρέμενε περίπου στα 7.105 MiB. Επιβεβαιώθηκε ότι η `OLLAMA_KEEP_ALIVE=24h` λειτουργεί όπως αναμένεται: το Ollama δεν αποφορτώνει το μοντέλο μετά από σύντομη περίοδο αδράνειας. Επομένως, ένα επόμενο αίτημα που δρομολογείται στο συγκεκριμένο pod δεν χρειάζεται να επιβαρυνθεί εκ νέου με το κόστος φόρτωσης του μοντέλου στην κύρια μνήμη.

Σενάριο Β — συνεχής φόρτος (περιορισμός της ρύθμισης): Αντίθετα, μια παρατεταμένη δοκιμή συνεχούς φόρτου με 30 ταυτόχρονους χρήστες και διάρκεια άνω των 30 λεπτών εμφάνισε ποσοστό αποτυχιών περίπου 14,9% μετά από 12–27 λεπτά, με 3 από τα 5 pods να τερματίζονται λόγω OOMKill. Το αποτέλεσμα αυτό συνδέθηκε με αυξημένη πίεση μνήμης, καθώς το μοντέλο παρέμενε φορτωμένο στην κύρια μνήμη ενώ εξυπηρετούνταν συνεχώς αιτήματα. Η διατήρηση του μοντέλου, σε συνδυασμό με την ταυτόχρονη κατάσταση επεξεργασίας αιτημάτων, όπως η κρυφή μνήμη κλειδιών-τιμών (KV cache), μπορούσε να οδηγήσει σε υπέρβαση του ορίου μνήμης ανά pod.

Η αντιπαράθεση των δύο σεναρίων δείχνει ότι η `OLLAMA_KEEP_ALIVE` δεν είναι μια απλή ρύθμιση ενεργοποίησης ή απενεργοποίησης, αλλά μια παράμετρος με σημαντικές επιπτώσεις στη χρήση μνήμης. Για τον πειραματικό φόρτο της εργασίας, δηλαδή δοκιμές μικρής διάρκειας έως 15 λεπτών και έως 10 χρήστες, η τιμή 24h ήταν κατάλληλη. Σε αναπτύξεις παραγωγικής λειτουργίας με συνεχή υψηλό φόρτο, θα απαιτούνταν μικρότερος χρόνος διατήρησης του μοντέλου στη

μνήμη, αυστηρότερος περιορισμός της ουράς αιτημάτων ή περιοδική επανεκκίνηση των pods, ώστε να αποφεύγεται η ανεξέλεγκτη αύξηση της χρήσης μνήμης.

6.6 Σύγκριση Ποιότητας Απαντήσεων Phi-3 Mini vs Mistral 7B

Πέρα από τις μετρήσεις απόδοσης, δηλαδή την καθυστέρηση, τον ρυθμό εξυπηρέτησης και την κατανάλωση μνήμης, που παρουσιάστηκαν στην Ενότητα 6.3, ένα εξίσου σημαντικό ερώτημα είναι αν η ταχύτερη απόκριση του Phi-3 Mini συνοδεύεται από υποβάθμιση της ποιότητας των απαντήσεων. Στην ενότητα αυτή παρουσιάζεται ένα επαληθευμένο εμπειρικό παράδειγμα από τις δοκιμές της παρούσας εργασίας, σε συνδυασμό με δημοσιευμένα αποτελέσματα αξιολόγησης από τη σχετική βιβλιογραφία [3], [4].

6.6.1 Εμπειρική Σύγκριση

Για την ενδεικτική ποιοτική αξιολόγηση χρησιμοποιήθηκε ένα σύνολο ερωτημάτων ανάκτησης πραγματολογικών πληροφοριών (factual retrieval) πάνω στη γνωσιακή βάση Atlas Systems. Τα ερωτήματα κάλυπταν έργα, υπαλλήλους, πολιτικές προσωπικού, συγκρίσεις έργων και επακόλουθα ερωτήματα (follow-up questions). Οι απαντήσεις αξιολογήθηκαν ως προς την ορθότητα, την πληρότητα και την ικανότητα διατήρησης συμφραζομένων.

Κατηγορία	Phi-3 Mini (3.8B)	Mistral 7B (7B)
Factual Retrieval	Ορθή απάντηση	Ορθή απάντηση
Policy Retrieval	Ορθή απάντηση	Ορθή απάντηση
Project Comparison	Ορθή απάντηση	Ορθή απάντηση
List Extraction	Μερικώς πλήρης απαρίθμηση	Πληρέστερη απαρίθμηση

Κατηγορία	Phi-3 Mini (3.8B)	Mistral 7B (7B)
Missing Information Handling	Περιστασιακή κατασκευή πληροφορίας (hallucination)	Αναγνώριση έλλειψης πληροφορίας
Follow-up Questions	Μερική διατήρηση συμφραζομένου	Πληρέστερη διατήρηση συμφραζομένου

Πίνακας 4 Ποιοτική Σύγκριση Phi-3 Mini και Mistral 7B

Τα δύο μοντέλα παρουσίασαν παρόμοια συμπεριφορά σε ερωτήματα απλής ανάκτησης πληροφοριών, όπως προϋπολογισμοί έργων, χρονοδιαγράμματα και πολιτικές προσωπικού. Οι σημαντικότερες διαφορές εμφανίστηκαν σε περιπτώσεις όπου η ζητούμενη πληροφορία δεν υπήρχε ρητά στα ανακτημένα έγγραφα ή όταν απαιτούνταν διατήρηση συμφραζομένων μεταξύ διαδοχικών ερωτήσεων. Σε αυτές τις περιπτώσεις, το Mistral 7B παρουσίασε μεγαλύτερη συνέπεια στον χειρισμό ελλιπούς πληροφορίας και πληρέστερη διατήρηση συμφραζομένων, ενώ το Phi-3 Mini εμφάνισε περιστασιακά τάση παραγωγής μη τεκμηριωμένων συμπερασμάτων. Παρ' όλα αυτά, η συνολική ποιότητα των απαντήσεων του Phi-3 Mini κρίθηκε επαρκής για τη συγκεκριμένη γνωσιακή βάση και για φόρτους εργασίας RAG που βασίζονται κυρίως στην ανάκτηση πραγματολογικών πληροφοριών.

6.6.2 Δημοσιευμένες Δοκιμασίες Αξιολόγησης

Η εμπειρική παρατήρηση τοποθετείται σε ευρύτερο πλαίσιο μέσω των δημοσιευμένων benchmarks από τα technical reports των δύο μοντέλων. Ο Πίνακας 6.4 συνοψίζει τα κύρια αποτελέσματα από το Phi-3 Technical Report της Microsoft Research [3]:

Benchmark	Phi-3 Mini (3.8B)	Mistral 7B
MMLU (5 παραδείγματα)	68.8%	61.7%
HellaSwag (5 παραδείγματα)	76.7%	58,5%
GSM8K (8 παραδείγματα, αλυσίδα σκέψης)	82.5%	46.4%
ANLI (7 παραδείγματα)	52.8%	47.1%

Πίνακας 5 Δημοσιευμένα benchmark scores Phi-3 Mini vs Mistral 7B (Phi-3 Technical Report, Microsoft 2024 [3]).

Σύμφωνα με τα παραπάνω αποτελέσματα, το Phi-3 Mini υπερέχει του Mistral 7B και στις τέσσερις επιλεγμένες δοκιμασίες, παρά το γεγονός ότι διαθέτει σχεδόν τις μισές παραμέτρους. Η διαφορά είναι ιδιαίτερα έντονη στο GSM8K, όπου το Phi-3 Mini επιτυγχάνει 82,5% έναντι 46,4%, δηλαδή διαφορά 36,1 ποσοστιαίων μονάδων. Στη δοκιμασία γενικής γνώσης MMLU, η διαφορά είναι 7,1 ποσοστιαίες μονάδες.

Από την άλλη πλευρά, το τεχνικό κείμενο του Mistral 7B [4] αναδεικνύει ιδιαίτερα τις επιδόσεις του μοντέλου σε εργασίες παραγωγής κώδικα, όπου προσεγγίζει την απόδοση του εξειδικευμένου Code Llama 7B. Τα παραπάνω αποτελέσματα δεν αποτελούν άμεση αξιολόγηση της ποιότητας του συστήματος RAG της παρούσας εργασίας, αλλά παρέχουν ευρύτερο πλαίσιο για την ερμηνεία της εμπειρικής σύγκρισης.

6.6.3 Σύνθεση και Σύσταση

Συνδυάζοντας την εμπειρική παρατήρηση με τα δημοσιευμένα αποτελέσματα αξιολόγησης, προκύπτει μια χαρακτηριστική διαφοροποίηση. Από τη μία πλευρά, τα δημοσιευμένα αποτελέσματα δείχνουν υπεροχή του Phi-3 Mini στις επιλεγμένες δοκιμασίες συλλογισμού, γενικής γνώσης και μαθηματικών. Από την άλλη, η

εμπειρική αξιολόγηση ανέδειξε συγκεκριμένες περιπτώσεις όπου το Mistral 7B παρείχε πληρέστερες απαντήσεις: στην εξαντλητική απαρίθμηση λιστών, στον χειρισμό ελλιπούς πληροφορίας και στη διατήρηση συμφραζομένων σε επακόλουθα ερωτήματα.

Μια πιθανή ερμηνεία είναι ότι το Phi-3 Mini τείνει να παράγει πιο σύντομες και συνοπτικές απαντήσεις, ενώ το Mistral 7B αναπτύσσει εκτενέστερα το διαθέσιμο πλαίσιο. Η συμπεριφορά αυτή είναι θετική στις περιπτώσεις όπου απαιτείται πλήρης απαρίθμηση ή προσεκτική διαχείριση πληροφορίας που δεν υπάρχει στα ανακτημένα έγγραφα, αλλά συνεπάγεται μεγαλύτερο υπολογιστικό κόστος στην εξεταζόμενη υποδομή CPU.

Για τον συγκεκριμένο φόρτο εργασίας της παρούσας εργασίας, δηλαδή ανάκτηση πραγματολογικών πληροφοριών από δομημένη εταιρική γνωσιακή βάση, η σύσταση είναι υπέρ του Phi-3 Mini. Παρέχει επαρκή ποιότητα απαντήσεων για τις περισσότερες κατηγορίες ερωτημάτων, με σημαντικά καλύτερη απόδοση και χωρητικότητα: 3,34–3,69 φορές χαμηλότερη διάμεση καθυστέρηση, 3–5 φορές υψηλότερο ρυθμό εξυπηρέτησης και 25–42% χαμηλότερη κατανάλωση μνήμης.

Ωστόσο, η επιλογή αυτή δεν είναι καθολική. Σε εφαρμογές όπου η πληρότητα λιστών, η αυστηρή αναγνώριση ελλιπούς πληροφορίας ή οι εκτενείς συνομιλίες πολλαπλών γύρων αποτελούν βασικές απαιτήσεις, το Mistral 7B μπορεί να είναι καταλληλότερο, εφόσον η υποδομή μπορεί να υποστηρίξει το αυξημένο κόστος του. Για περιπτώσεις απαρίθμησης, μέρος της διαφοράς μπορεί να μετριαστεί σε επίπεδο RAG μέσω δομημένης ανάκτησης και μετεπεξεργασίας των αποτελεσμάτων, χωρίς να απαιτείται απαραίτητα η χρήση μεγαλύτερου μοντέλου.

6.7 Πρακτικά Σημεία Προσοχής σε Παραγωγική Λειτουργία (Production Gotchas)

Κατά την εφαρμογή του μηχανισμού προθέρμανσης μέσω DaemonSet, προέκυψαν και λειτουργικά ζητήματα, όπως περιορισμοί στη λήψη εικόνων από το μητρώο Docker Hub και ο αυξημένος χρόνος ενημέρωσης

των rods σε όλους τους κόμβους. Τα ζητήματα αυτά αντιμετωπίστηκαν με χρήση πιστοποιημένων λήψεων εικόνων και παράλληλη ενημέρωση του DaemonSet, χωρίς να επηρεάζουν τα κύρια πειράματα της εργασίας.

6.8 Σύνοψη Παρατηρήσεων και Περιορισμοί

- Όλες οι μετρήσεις πραγματοποιήθηκαν σε υποδομή αποκλειστικής χρήσης CPU. Η συμπεριφορά σε υποδομή GPU θα διέφερε ποσοτικά και θα απαιτούσε προσαρμογές, ωστόσο ζητήματα όπως η ψυχρή εκκίνηση, η κλιμάκωση προς τα πάνω και η διαχείριση πόρων παραμένουν σημαντικά.
- Το vndcloud είναι κοινόχρηστο υπολογιστικό σύμπλεγμα. Ορισμένες μετρήσεις ενδέχεται να επηρεάστηκαν από φόρτους εργασίας άλλων χρηστών.
- Η γνωσιακή βάση είναι συνθετική. Σε πραγματική εταιρική ανάπτυξη θα απαιτούνταν επιπλέον μέριμνα για την προστασία δεδομένων και την επιλογή του μοντέλου διανυσματικών αναπαραστάσεων.
- Οι δοκιμές φόρτου είχαν διάρκεια 10–15 λεπτών ανά εκτέλεση. Μακρόχρονες δοκιμές διάρκειας ωρών θα μπορούσαν να αναδείξουν πρόσθετα φαινόμενα, όπως αύξηση της κατανάλωσης μνήμης ή σταδιακή υποβάθμιση της απόδοσης, τα οποία δεν παρατηρήθηκαν στα παρόντα χρονικά παράθυρα.

ΚΕΦΑΛΑΙΟ 7 Συμπεράσματα και Μελλοντική Εργασία

7.1 Σύνοψη Βασικών Ευρημάτων

Η εργασία αυτή σχεδίασε, υλοποίησε και αξιολόγησε ένα ολοκληρωμένο σύστημα RAG σε πειραματική υποδομή Kubernetes. Τα κύρια ευρήματα συνοψίζονται ως εξής:

- Η ψυχρή εκκίνηση ενός LLM δεν περιορίζεται μόνο στη δημιουργία του pod· περιλαμβάνει τη λήψη της εικόνας περιέκτη, τη διαθεσιμότητα του μοντέλου στον δίσκο και τη φόρτωσή του στην κύρια μνήμη. Κάθε στάδιο πρέπει να αντιμετωπίζεται ξεχωριστά, ώστε ο μετριάσμός της καθυστέρησης να είναι αποτελεσματικός.
- Ένα pod σε κατάσταση Running δεν σημαίνει απαραίτητα ότι είναι έτοιμο να εξυπηρετήσει αιτήματα. Ο συνδυασμός μηχανισμού postStart και προσαρμοσμένου ανιχνευτή ετοιμότητας συμβάλλει ώστε ένα νέο pod να μην δέχεται αιτήματα πριν ολοκληρωθεί η απαραίτητη προετοιμασία του μοντέλου.
- Η χρήση προδημιουργημένων εικόνων Ollama σε συνδυασμό με προθέρμανση κόμβων μέσω DaemonSet μειώνει σημαντικά τους χρόνους εκκίνησης νέων αντιγράφων, εξαλείφοντας τη λήψη της εικόνας και του μοντέλου όταν αυτά είναι ήδη διαθέσιμα στον κόμβο. Με την επιπρόσθετη διόρθωση της σημαίας --force στον περιέκτη αρχικοποίησης, ο χρόνος ψυχρής εκκίνησης του Mistral σε κόμβο με ήδη διαθέσιμη εικόνα του Backend μειώθηκε από περίπου 6,5 λεπτά σε περίπου 2,8 λεπτά.
- Στην άμεση συγκριτική αξιολόγηση σε υποδομή CPU, το Phi-3 Mini παρουσίασε σημαντικά καλύτερη απόδοση από το Mistral 7B. Στις δοκιμές με 3 και 5 χρήστες είχε 3,34–3,69 φορές χαμηλότερη διάμεση καθυστέρηση, 3–5 φορές υψηλότερο ρυθμό εξυπηρέτησης και 25–42% χαμηλότερη κατανάλωση μνήμης. Και τα δύο μοντέλα διατήρησαν ποσοστό αποτυχιών 0% έως και τους 10 ταυτόχρονους χρήστες, όμως η

καθυστέρηση p95 του Mistral στους 10 χρήστες έφτασε τα 13,85 λεπτά, έναντι 4,4 λεπτών για το Phi-3 Mini.

- Στην ποιοτική αξιολόγηση καλύφθηκαν έξι κατηγορίες ερωτημάτων: ανάκτηση πραγματολογικών πληροφοριών, ανάκτηση πολιτικών, σύγκριση έργων, εξαγωγή στοιχείων λίστας, χειρισμός ελλιπούς πληροφορίας και επακόλουθα ερωτήματα. Σε ερωτήματα απλής ανάκτησης πληροφοριών, και τα δύο μοντέλα απάντησαν ορθά. Σε ερωτήματα όπου η πληροφορία δεν υπήρχε ρητά στα ανακτημένα τμήματα ή απαιτούνταν διατήρηση συμφραζομένων, το Mistral 7B παρουσίασε μεγαλύτερη συνέπεια, ενώ το Phi-3 Mini εμφάνισε περιστασιακά τάση παραγωγής μη τεκμηριωμένης πληροφορίας και μερική διατήρηση συμφραζομένων.

- Η ποιότητα των απαντήσεων του RAG εξαρτάται σε μεγάλο βαθμό από τη δομή των δεδομένων, την ποιότητα των μεταδεδομένων και την αποτελεσματικότητα της ανάκτησης, όχι μόνο από την επιλογή του LLM. Αντιφάσεις ή διπλές αναπαραστάσεις πληροφορίας στη γνωσιακή βάση μπορούν να προκαλέσουν λανθασμένες απαντήσεις, ακόμη και όταν ανακτώνται σχετικά τμήματα.

- Για τον συγκεκριμένο φόρτο εργασίας, δηλαδή την ανάκτηση πραγματολογικών πληροφοριών από δομημένη εταιρική γνωσιακή βάση σε υποδομή CPU, το Phi-3 Mini αποτελεί την καταλληλότερη επιλογή. Συνδυάζει επαρκή ποιότητα απαντήσεων με σημαντικά καλύτερη απόδοση σε χρόνο και χωρητικότητα και υπεροχή στις επιλεγμένες δημοσιευμένες δοκιμασίες αξιολόγησης. Ωστόσο, σε εφαρμογές όπου η πληρότητα λιστών, η αυστηρή αναγνώριση ελλιπούς πληροφορίας ή οι συνομιλίες πολλαπλών γύρων είναι κρίσιμες απαιτήσεις, το Mistral 7B μπορεί να είναι καταλληλότερο, εφόσον η διαθέσιμη υποδομή υποστηρίζει το αυξημένο υπολογιστικό του κόστος.

7.2 Διδάγματα Παραγωγής

Πέρα από τα κύρια ευρήματα, η εργασία ανέδειξε αρκετά διδάγματα χρήσιμα για αναπτύξεις σε περιβάλλοντα παραγωγικής λειτουργίας:

- Ο HPA μπορεί να είναι μηχανικά σωστός αλλά συστημικά ανεπαρκής. Η διάκριση μεταξύ της ενεργοποίησης του μηχανισμού ελέγχου και της στιγμής κατά την οποία η νέα χωρητικότητα είναι πραγματικά διαθέσιμη είναι κρίσιμη, ώστε η απλή δημιουργία νέων αντιγράφων να μη θεωρείται από μόνη της επιτυχής κλιμάκωση.
- Σε υποδομές αποκλειστικής χρήσης CPU, η ψυχρή εκκίνηση είναι ιδιαίτερα επιβαρυντική ως προς τον χρόνο και τους διαθέσιμους πόρους. Ένα νέο αντίγραφο που δεν έχει ολοκληρώσει την προετοιμασία του δεν συνεισφέρει ακόμη στην εξυπηρέτηση αιτημάτων, ενώ ήδη καταναλώνει πόρους του συστήματος.
- Η λειτουργική αξιοπιστία του συστήματος δεν εξαρτάται μόνο από την αρχιτεκτονική του εφαρμογής, αλλά και από ζητήματα υποδομής, όπως η διαθεσιμότητα εικόνων περιέκτη, ο χώρος αποθήκευσης και η συμπεριφορά του δικτύου. Η καταγραφή και η αντιμετώπιση αυτών των ζητημάτων είναι απαραίτητες για την αξιόπιστη λειτουργία ενός συστήματος RAG υπό αυτόματη κλιμάκωση

7.3 Μελλοντικές Κατευθύνσεις

Η υλοποίηση επιτρέπει αρκετές επεκτάσεις:

- Επιτάχυνση με GPU: Η μετάβαση σε υποδομή GPU θα απαιτούσε κόμβους με διαθέσιμη GPU, κατάλληλους οδηγούς και μηχανισμό διάθεσης συσκευών στο Kubernetes, καθώς και εικόνα Ollama με υποστήριξη GPU. Η GPU θα δηλωνόταν ως πόρος στο πεδίο limits, για παράδειγμα `nvidia.com/gpu`: 1. Τα μοτίβα μετριάσμου της ψυχρής εκκίνησης που αναπτύχθηκαν παραμένουν εφαρμόσιμα.
- Κλιμάκωση βάσει γεγονότων με KEDA: Αντί της αποκλειστικής κλιμάκωσης βάσει χρήσης CPU, θα μπορούσε να χρησιμοποιηθεί κλιμάκωση βάσει ρυθμού αιτημάτων, αριθμού ταυτόχρονων αιτημάτων ή μήκους ουράς. Η προσέγγιση αυτή θα μπορούσε να ενεργοποιεί την

κλιμάκωση νωρίτερα από έναν μηχανισμό που βασίζεται μόνο στη χρήση CPU.

- Αποδοτικότερη εξυπηρέτηση μοντέλων με vLLM: Σε μελλοντική υποδομή GPU, η χρήση εξυπηρετητή μοντέλων όπως το vLLM θα μπορούσε να αξιοποιήσει συνεχή ομαδοποίηση αιτημάτων, με στόχο τη βελτίωση του ρυθμού εξυπηρέτησης και της αξιοποίησης της GPU.
- Δρομολόγηση αιτημάτων επιπέδου εφαρμογής: Η χρήση μηχανισμού δρομολόγησης επιπέδου L7, όπως το Istio, θα μπορούσε να επιτρέψει επιλογή αντιγράφων με βάση τον μικρότερο αριθμό ενεργών αιτημάτων. Η προσέγγιση αυτή θα άξιζε να αξιολογηθεί σε μελλοντικές δοκιμές, εφόσον επιβεβαιωθεί μη ομοιόμορφη κατανομή της κίνησης μεταξύ των pods.

Βιβλιογραφία

- [1] Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS 2017). arXiv:1706.03762.
- [2] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS 2020). arXiv:2005.11401.
- [3] Abdin, M., et al. (2024). *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. arXiv:2404.14219.
- [4] Jiang, A., et al. (2023). *Mistral 7B*. Mistral AI. <https://mistral.ai/news/announcing-mistral-7b/>
- [5] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. Communications of the ACM, 59(5).

- [6] Kubernetes Authors. *Horizontal Pod Autoscaler*. Kubernetes Documentation. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [7] Kwon, W., et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023). arXiv:2309.06180.
- [8] Gerganov, G. *llama.cpp: LLM Inference in C/C++*. <https://github.com/ggerganov/llama.cpp>
- [9] Ollama. *Get Up and Running with Large Language Models*. <https://ollama.com/>
- [10] Chroma. *Chroma: The AI-Native Open-Source Embedding Database*. <https://www.trychroma.com/>
- [11] KEDA. *Kubernetes-based Event Driven Autoscaling*. <https://keda.sh/>
- [12] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019). arXiv:1908.10084.
- [13] Wang, L., et al. (2024). *Multilingual E5 Text Embeddings: A Technical Report*. <https://huggingface.co/intfloat/multilingual-e5-small>